



UNIVERSITÉ
CAEN
NORMANDIE

CAMPUS 2
DÉPARTEMENT DE MATHÉMATIQUES
Master 2 Mathématiques et Applications : Statistiques
Appliquées et Analyse Décisionnelle

Dans le cadre du projet des travaux encadrés des recherches

Modélisation du Risque de Crédit

Hassan Alio Malik
Mbaitel Serge DJERAN
Zolifaga Samuel Yeo

Encadré par
M. Bertrand Maillot

8 mars 2026

Table des matières

1	Introduction	4
1.1	Contexte et justification du sujet	4
1.2	Problématique	4
1.3	Hypothèses de recherche	4
1.4	Objectifs de l'étude	4
1.4.1	Objectif général	4
1.4.2	Objectifs spécifiques	5
1.5	Méthodologie adoptée	5
2	CHAPITRE 1 : CADRE CONCEPTUEL ET THÉORIQUE DU RISQUE DE CRÉDIT	6
2.1	Définition du risque de crédit	6
2.2	Apport du Machine Learning dans l'évaluation du risque . . .	6
2.2.1	Apprentissage supervisé	6
2.2.2	Modèles d'ensemble	6
3	CHAPITRE 2 : PRÉTRAITEMENT ET INGÉNIERIE DES DONNÉES	7
3.1	Description des données	7
3.1.1	Base des prêts (loans.csv)	7
3.1.2	Base des clients (customers.csv)	8
3.1.3	Base du bureau de crédit (bureau_data.csv)	8
3.2	2.2. Chargement des données	9
3.3	Chargement des données	9
3.4	Transformation des unités monétaires	11
3.5	Fusion et structuration de la base consolidée	11
3.6	Conversion et encodage de la variable cible	16
3.7	Séparation des données : stratégie Train/Test	16
3.8	Gestion des valeurs manquantes	17
3.9	Détection et traitement des valeurs aberrantes	20
3.9.1	Analyse exploratoire par boxplots	20
3.10	Analyse des distributions et asymétrie	22
3.10.1	Suppression des valeurs manquantes : Processing Fee .	23
3.10.2	Utilisation d'autres règles de gestion pour la validation des données	25
3.11	Analyse et correction des variables catégorielles	25
3.11.1	Gestion des erreurs dans la colonne <code>loan_purpose</code> . . .	26
3.12	Analyse exploratoire des variables numériques	27
3.12.1	Colonne <code>age</code>	27

3.12.2	Analyse par densité (KDE) pour l'ensemble des variables numériques	28
3.13	Ingénierie des caractéristiques	30
3.13.1	Ratio Loan-to-Income (LTI)	30
3.13.2	Taux de délinquance (impayés)	31
3.13.3	Gravité moyenne des retards	33
3.14	Suppression des variables redondantes	34
3.15	Analyse de la multicolinéarité (VIF)	36
3.16	Même transformation sur les ensembles de test	37
3.17	Sélection des variables par Information Value (IV)	43
3.17.1	Sélection des variables selon l'Information Value	48
4	CHAPITRE 3 : MODÉLISATION ET ÉVALUATION DES PERFORMANCES	48
4.1	Encodage des variables	48
4.2	Stratégie expérimentale	49
4.3	Tentative 1 : Modélisation sans traitement du déséquilibre	49
4.3.1	Régression Logistique	49
4.3.2	Random Forest	50
4.3.3	XGBoost	51
4.3.4	Optimisation des hyperparamètres – RandomizedSearchCV	52
4.4	Tentative 2 : Traitement du déséquilibre des classes – Sous-échantillonnage (Random Under-Sampling)	54
4.5	4.5 Tentative 3 : SMOTE Tomek + Optimisation Optuna	56
4.5.1	4.5.1 SMOTE Tomek	56
4.5.2	4.5.2 Optimisation via Optuna	57
4.6	4.6 Tentative 4 : XGBoost avec optimisation par Optuna	60
4.6.1	Évaluation sur le jeu de test	63
4.7	Comparaison Logistique vs XGBoost	65
4.8	Évaluation approfondie du modèle final	65
4.8.1	Courbe ROC et AUC	65
4.8.2	Coefficient de Gini	67
4.8.3	Rank Ordering (Déciles)	68
4.8.4	Statistique KS	72
4.9	Importance des variables	73
4.10	Industrialisation	74
	Conclusion	76
	Conclusion	76

1 Introduction

1.1 Contexte et justification du sujet

Dans un contexte marqué par l'intensification de la bancarisation et la digitalisation rapide des services financiers, les établissements de crédit font face à une exigence opérationnelle forte : évaluer, en un temps très court, le risque associé à chaque demande de prêt.

1.2 Problématique

Malgré la disponibilité croissante des données clients et transactionnelles, la construction d'un modèle de scoring performant soulève plusieurs défis méthodologiques et opérationnels :

- Comment intégrer efficacement des sources de données hétérogènes ?
- Comment traiter le déséquilibre naturel entre clients en défaut et non-défaut ?
- Quels algorithmes offrent le meilleur compromis entre performance et interprétabilité ?
- Comment transformer une probabilité de défaut en score bancaire exploitable ?

1.3 Hypothèses de recherche

Les hypothèses structurant ce travail sont les suivantes :

1. Les variables comportementales ont un pouvoir prédictif supérieur.
2. Les modèles d'ensemble sont plus performants que les modèles linéaires.
3. Un prétraitement rigoureux améliore la généralisation.
4. Il est possible de construire un score interprétable.

1.4 Objectifs de l'étude

1.4.1 Objectif général

Développer et évaluer un modèle de prédiction du risque de défaut basé sur le Machine Learning.

1.4.2 Objectifs spécifiques

- Fusionner plusieurs bases de données ;
- Effectuer un prétraitement rigoureux ;
- Réaliser une analyse exploratoire ;
- Comparer différents modèles ;
- Évaluer les performances ;
- Interpréter les résultats ;
- Proposer une segmentation des profils de risque.

1.5 Méthodologie adoptée

1. Compréhension des données ;
2. Prétraitement ;
3. Modélisation ;
4. Évaluation ;
5. Interprétation.

2 CHAPITRE 1 : CADRE CONCEPTUEL ET THÉORIQUE DU RISQUE DE CRÉDIT

2.1 Définition du risque de crédit

Le risque de crédit correspond à la probabilité qu'un emprunteur fasse défaut sur le remboursement d'un prêt. Dans le cadre de ce projet, l'accent est mis sur l'estimation de la probabilité de défaut à travers une approche de classification binaire.

2.2 Apport du Machine Learning dans l'évaluation du risque

L'apprentissage automatique permet de détecter des structures complexes dans les données sans imposer d'hypothèses fortes sur leur distribution.

2.2.1 Apprentissage supervisé

Le problème étudié relève de la classification supervisée binaire : à partir d'observations étiquetées (défaut / non-défaut), le modèle apprend une fonction permettant de prédire la classe d'un nouvel individu.

2.2.2 Modèles d'ensemble

Les méthodes comme les forêts aléatoires reposent sur la combinaison de plusieurs arbres de décision afin de réduire la variance et améliorer la performance prédictive.

Ces modèles présentent plusieurs avantages :

- Capacité à modéliser des relations non linéaires ;
- Robustesse aux valeurs aberrantes ;
- Gestion automatique des interactions entre variables.

3 CHAPITRE 2 : PRÉTRAITEMENT ET INGÉNIERIE DES DONNÉES

3.1 Description des données

Le projet repose sur trois bases de données distinctes, chacune décrivant une dimension spécifique du risque de crédit : les caractéristiques du prêt, le profil du client et son historique de crédit.

3.1.1 Base des prêts (loans.csv)

Cette base contient les informations relatives aux prêts octroyés par l'institution financière. Ces variables sont principalement financières et contractuelles. Elles permettent d'analyser la structure du crédit accordé et son exposition au risque.

Colonne	Signification
loan_id	ID unique du prêt
cust_id	ID unique du client
loan_purpose	Objectif : Home, Auto, Personal, Education
loan_type	Type : Secured (garanti), Unsecured
sanction_amount	Montant approuvé (INR)
loan_amount	Montant emprunté (INR)
processing_fee	Frais de traitement
gst	Taxe GST (18%)
net_disbursement	Montant net versé
loan_tenure_months	Durée en mois
principal_outstanding	Capital restant dû
bank_balance_at_application	Solde bancaire à la demande
disbursal_date	Date déblocage prêt
installment_start_dt	Date 1ère échéance
default	Cible : 1 = défaut, 0 = OK

TABLE 1 – Description des variables de la base des prêts (loans.csv)

3.1.2 Base des clients (customers.csv)

Cette base regroupe les caractéristiques socio-démographiques des emprunteurs : Ces variables permettent de caractériser le profil individuel du demandeur et sont traditionnellement utilisées dans les systèmes de scoring bancaire.

Colonne	Signification
cust_id	ID unique du client
age	Âge
gender	Genre
experience	Années expérience pro
marital_status	Etat civil, Marié/Célibataire
employment_status	Statut professionnel du client
income	Revenu annuel
number_of_dependants	Nombre de personnes à charge du client
residence_type	Type de résidence
years_at_current_address	nombre d'années passées à l'adresse actuelle.
city	Ville
state	État ou région de résidence du client
zipcode	Code postal de l'adresse du client

Description des variables de la base des prêts (customers.csv)

3.1.3 Base du bureau de crédit (bureau_data.csv)

Cette troisième base contient des informations relatives au comportement financier passé du client : Ces variables comportementales sont particulièrement importantes, car elles reflètent la discipline de remboursement passée du client, généralement considérée comme un fort indicateur de risque futur.

Colonne	Signification
cust_id	ID unique du client
number_of_open_accounts	Nombre de comptes de crédit ouverts par le client
number_of_closed_accounts	Nombre de comptes de crédit fermés par le client.
total_loan_months	Nombre total de mois pendant lesquels le client a eu des prêts
delinquent_months	Nombre de mois durant lesquels le client a eu des retards de paiement
total_dpd	Nombre total de jours de retard accumulés sur les paiements
enquiry_count	Nombre de demandes de crédit effectuées par le client
credit_utilization_ratio	Ratio de l'utilisation du crédit

Description des variables de la base des prêts (bureau_data.csv)

3.2 2.2. Chargement des données

3.3 Chargement des données

```

1 import pandas as pd
2 import numpy as np
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5 from sklearn.model_selection import train_test_split
6
7 pd.set_option("display.max_columns", 100)
8
9 loans = pd.read_csv("loans.csv")
10 customers = pd.read_csv("customers.csv")
11 bureau = pd.read_csv("bureau_data.csv")
12
13 print("loans :", loans.shape)
14 print("customers :", customers.shape)
15 print("bureau :", bureau.shape)
16
17 loans.head()
```

```

loans : (50000, 15)
customers : (50000, 12)
bureau : (50000, 8)
```

	loan_id	cust_id	loan_purpose	loan_type	sanction_amount	loan_amount	processing_fee	gst	net_disbursement	loan_tenure_months
0	L00001	C00001	Auto	Secured	3004000	2467000	49340.0	444060	1973600	33
1	L00002	C00002	Home	Secured	4161000	3883000	77660.0	698940	3106400	30
2	L00003	C00003	Personal	Unsecured	2401000	2170000	43400.0	390600	1736000	21
3	L00004	C00004	Personal	Unsecured	2345000	1747000	34940.0	314460	1397600	6
4	L00005	C00005	Auto	Secured	4647000	4520000	90400.0	813600	3616000	28
	principal_outstanding	bank_balance_at_application	disbursal_date	instalment_start_dt	default					
	1630408		873386	2019-07-24	2019-08-10	False				
	709309		464100	2019-07-24	2019-08-15	False				
	1562399		1476042	2019-07-24	2019-08-21	False				
	1257839		1031094	2019-07-24	2019-08-09	False				
	1772334		1032458	2019-07-24	2019-08-02	False				

```
1 customers.head()
```

[91]:	cust_id	age	gender	marital_status	employment_status	income	number_of_dependants	residence_type	years_at_current_address	city	state	zipcode
0	C00001	44	M	Married	Self-Employed	2586000	3	Owned	27	Delhi	Delhi	110001
1	C00002	38	M	Married	Salaried	1206000	3	Owned	4	Chennai	Tamil Nadu	600001
2	C00003	46	F	Married	Self-Employed	2878000	3	Owned	24	Kolkata	West Bengal	700001
3	C00004	55	F	Single	Self-Employed	3547000	1	Owned	15	Bangalore	Karnataka	560001
4	C00005	37	M	Married	Salaried	3432000	3	Owned	28	Pune	Maharashtra	411001

```
1 customers.columns
```

```
Index(['cust_id', 'number_of_open_accounts', 'number_of_closed_accounts',
      'total_loan_months', 'delinquent_months', 'total_dpd',
      'enquiry_count',
      'credit_utilization_ratio'],
      dtype='object')
```

```
1 bureau.head()
```

[93]:	cust_id	number_of_open_accounts	number_of_closed_accounts	total_loan_months	delinquent_months	total_dpd	enquiry_count	credit_utilization_ratio
0	C00001	1	1	42	0	0	3	7
1	C00002	3	1	96	12	60	5	4
2	C00003	2	1	82	24	147	6	58
3	C00004	3	0	115	15	87	5	26
4	C00005	4	2	120	0	0	5	10

```
1 bureau.columns
```

```
Index(['cust_id', 'number_of_open_accounts', '
      number_of_closed_accounts',
      'total_loan_months', 'delinquent_months', 'total_dpd',
      'enquiry_count',
      'credit_utilization_ratio'],
      dtype='object')
```

```
1 loans.columns
```

```
Index(['loan_id', 'cust_id', 'loan_purpose', 'loan_type', '
      sanction_amount',
      'loan_amount', 'processing_fee', 'gst', '
      net_disbursement',
      'loan_tenure_months', 'principal_outstanding',
      'bank_balance_at_application', 'disbursal_date', '
      installment_start_dt',
      'default'],
      dtype='object')
```

3.4 Transformation des unités monétaires

Les montants financiers initialement exprimés en devise locale ont été convertis en euros afin d'assurer une cohérence d'interprétation économique et une meilleure comparabilité des variables monétaires.

```
1 conversion_rate = 0.011
```

```
1 customers['income'] = customers['income'] * conversion_rate
```

```
1 columns_to_convert = ['sanction_amount', 'loan_amount', '
      processing_fee', 'gst', 'net_disbursement', '
      principal_outstanding', 'bank_balance_at_application']
```

```
1 for col in columns_to_convert:
2     loans[col] = loans[col] * conversion_rate
```

3.5 Fusion et structuration de la base consolidée

Les trois bases de données initiales ont été fusionnées à l'aide de la clé primaire commune (cust_id) afin d'obtenir une base unique.

```
1 df = pd.merge(customers, loans, on='cust_id')
2 df.head(3)
```

[104]:

	cust_id	age	gender	marital_status	employment_status	income	number_of_dependants	residence_type	years_at_current_address
0	C00001	44	M	Married	Self-Employed	28446.0	3	Owned	27
1	C00002	38	M	Married	Salaried	13266.0	3	Owned	4
2	C00003	46	F	Married	Self-Employed	31658.0	3	Owned	24

city	state	zipcode	loan_id	loan_purpose	loan_type	sanction_amount	loan_amount	processing_fee	gst	net_disbursement	loan_tenure_months
Delhi	Delhi	110001	L00001	Auto	Secured	33044.0	27137.0	542.74	4884.66	21709.6	33
Chennai	Tamil Nadu	600001	L00002	Home	Secured	45771.0	42713.0	854.26	7688.34	34170.4	30
Kolkata	West Bengal	700001	L00003	Personal	Unsecured	26411.0	23870.0	477.40	4296.60	19096.0	21

principal_outstanding	bank_balance_at_application	disbursal_date	installment_start_dt	default
17934.488	9607.246	2019-07-24	2019-08-10	False
7802.399	5105.100	2019-07-24	2019-08-15	False
17186.389	16236.462	2019-07-24	2019-08-21	False

```
1 df.info
```

```

   cust_id  age  gender  marital_status  employment_status
0  C00001   44     M      Married      Self-Employed
   income \
1  C00002   38     M      Married      Salaried
   13266.0
2  C00003   46     F      Married      Self-Employed
   31658.0
3  C00004   55     F      Single      Self-Employed
   39017.0
4  C00005   37     M      Married      Salaried
   37752.0
...
49995  C49996  40     F      Single      Salaried
   8525.0
49996  C49997  39     M      Single      Salaried
   34287.0
49997  C49998  45     F      Single      Self-Employed
   14619.0
49998  C49999  42     F      Single      Self-Employed
   5863.0
49999  C50000  55     M      Married      Self-Employed
   34584.0

```

	number_of_dependants	residence_type	years_at_current_address	\
0	3	Owned	27	
1	3	Owned	4	
2	3	Owned	24	
3	1	Owned	15	
4	3	Owned	28	
...				
49995	2	Owned	11	
49996	0	Owned	9	
49997	0	Rented	27	
49998	2	Mortgaged	20	
49999	3	Owned	19	

	city	state	zipcode	loan_id	loan_purpose
	loan_type				
0	Delhi	Delhi	110001	L00001	Auto
	Secured				
1	Chennai	Tamil Nadu	600001	L00002	Home
	Secured				
2	Kolkata	West Bengal	700001	L00003	Personal
	Unsecured				
3	Bangalore	Karnataka	560001	L00004	Personal
	Unsecured				
4	Pune	Maharashtra	411001	L00005	Auto
	Secured				
...					
49995	Chennai	Tamil Nadu	600001	L49996	Personal
	Unsecured				
49996	Kolkata	West Bengal	700001	L49997	Auto
	Secured				
49997	Bangalore	Karnataka	560001	L49998	Home
	Secured				
49998	Hyderabad	Telangana	500001	L49999	Home
	Secured				
49999	Mumbai	Maharashtra	400001	L50000	Home
	Secured				

	sanction_amount	loan_amount	processing_fee	gst	\
0	33044.0	27137.0	542.74	4884.66	
1	45771.0	42713.0	854.26	7688.34	
2	26411.0	23870.0	477.40	4296.60	
3	25795.0	19217.0	384.34	3459.06	
4	51117.0	49720.0	994.40	8949.60	
...					
49995	6710.0	5885.0	117.70	1059.30	
49996	46321.0	35673.0	713.46	6421.14	
49997	45067.0	41140.0	822.80	7405.20	

49998	20581.0	17930.0	358.60	3227.40
49999	91058.0	68926.0	1378.52	12406.68
net_disbursement loan_tenure_months principal_outstanding \				
0	21709.6		33	
	17934.488			
1	34170.4		30	
	7802.399			
2	19096.0		21	
	17186.389			
3	15373.6		6	
	13836.229			
4	39776.0		28	
	19495.674			
...				
49995	4708.0		22	
	4237.189			
49996	28538.4		15	
	18221.324			
49997	32912.0		37	
	10039.601			
49998	14344.0		37	
	3300.385			
49999	55140.8		38	
	17816.579			
bank_balance_at_application disbursal_date installment_start_dt				
0		9607.246	2019-07-24	
	2019-08-10			
1		5105.100	2019-07-24	
	2019-08-15			
2		16236.462	2019-07-24	
	2019-08-21			
3		11342.034	2019-07-24	
	2019-08-09			
4		11357.038	2019-07-24	
	2019-08-02			
...				
49995		1963.170	2024-07-22	
	2024-08-15			
49996		11448.085	2024-07-22	
	2024-07-29			
49997		3910.071	2024-07-22	
	2024-07-25			
49998		1973.983	2024-07-22	
	2024-07-29			
49999		15002.614	2024-07-22	

```
2024-08-08

default
0    False
1    False
2    False
3    False
4    False
...
49995 False
49996 False
49997 False
49998 False
49999 False

[50000 rows x 26 columns]
```

```
1 df = pd.merge(df, bureau, on='cust_id')
2 df.head(3)
```

[106]:	cust_id	age	gender	marital_status	employment_status	income	number_of_dependants	residence_type	years_at_current_address	city	state	zipcode
0	C00001	44	M	Married	Self-Employed	28446.0	3	Owned	27	Delhi	Delhi	110001
1	C00002	38	M	Married	Salaried	13266.0	3	Owned	4	Chennai	Tamil Nadu	600001
2	C00003	46	F	Married	Self-Employed	31658.0	3	Owned	24	Kolkata	West Bengal	700001

loan_id	loan_purpose	loan_type	sanction_amount	loan_amount	processing_fee	gst	net_disbursement	loan_tenure_months	principal_outstanding
L00001	Auto	Secured	33044.0	27137.0	542.74	4884.66	21709.6	33	17934.488
L00002	Home	Secured	45771.0	42713.0	854.26	7688.34	34170.4	30	7802.399
L00003	Personal	Unsecured	26411.0	23870.0	477.40	4296.60	19096.0	21	17186.389

principal_outstanding	bank_balance_at_application	disbursal_date	installment_start_dt	default	number_of_open_accounts
17934.488	9607.246	2019-07-24	2019-08-10	False	1
7802.399	5105.100	2019-07-24	2019-08-15	False	3
17186.389	16236.462	2019-07-24	2019-08-21	False	2

number_of_closed_accounts	total_loan_months	delinquent_months	total_dpd	enquiry_count	credit_utilization_ratio
1	42	0	0	3	7
1	96	12	60	5	4
1	82	24	147	6	58

La base finale contient 50 000 individus et 33 variables explicatives et cibles.

3.6 Conversion et encodage de la variable cible

La variable `default`, initialement codée sous forme booléenne, a été convertie en variable binaire numérique :

- 0 : absence de défaut
- 1 : défaut

```
1 df['default'] = df['default'].astype(int)
2 df.default.value_counts()
```

```
default
0      45703
1       4297
Name: count, dtype: int64
```

On observe qu'il y a 45 703 clients n'ont pas eu de défaut de remboursement (`default = 0`), contre 4 297 clients en défaut (`default = 1`). Cela représente environ 91,4 % de non-défaut et 8,6 % de défaut.

3.7 Séparation des données : stratégie Train/Test

On effectue un *train/test split* afin de vérifier que le modèle généralise correctement et ne se contente pas de « mémoriser » les données d'apprentissage, dans le but d'éviter le surapprentissage.

```
1 X = df.drop("default", axis="columns")
2 y = df['default']
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, stratify=y, test_size=0.25, random_state=42
6 )
7
8 df_train = pd.concat([X_train, y_train], axis="columns")
9 df_test = pd.concat([X_test, y_test], axis="columns")
10
11 df_train.head(3)
```

```
[112]:
```

	cust_id	age	gender	marital_status	employment_status	income	number_of_dependants	residence_type	years_at_current_address	city	state
12746	C12747	59	M	Married	Self-Employed	124597.0	3	Owned	30	Hyderabad	Telangana
32495	C32496	44	F	Single	Salaried	7865.0	0	Owned	27	Mumbai	Maharashtra
43675	C43676	38	M	Single	Salaried	35145.0	0	Mortgage	26	Chennai	Tamil Nadu

[112]:	zipcode	loan_id	loan_purpose	loan_type	sanction_amount	loan_amount	processing_fee	gst	net_disbursement	loan_tenure_months	principal_outstanding
	500001	L12747	Home	Secured	364331.0	257862.0	5157.24	46415.16	206289.6	28	55000.000
	400001	L32496	Education	Secured	12925.0	12639.0	252.78	2275.02	10111.2	50	5139.519
	600001	L43676	Home	Secured	125499.0	124256.0	2485.12	22366.08	99404.8	32	18224.503

number_of_open_accounts	number_of_closed_accounts	total_loan_months	delinquent_months	total_dpd	enquiry_count	credit_utilization_ratio	default
4	2	152	20	118	4	36	0
3	1	160	10	62	5	5	0
1	1	54	12	67	4	0	0

3.8 Gestion des valeurs manquantes

```
1 df_train.isna().sum()
```

```

cust_id          0
age              0
gender           0
marital_status   0
employment_status 0
income           0
number_of_dependants 0
residence_type    47
years_at_current_address 0
city              0
state             0
zipcode           0
loan_id           0
loan_purpose        0
loan_type         0
sanction_amount   0
loan_amount       0
processing_fee     0
gst               0
net_disbursement  0
loan_tenure_months 0
principal_outstanding 0
bank_balance_at_application 0
disbursal_date    0
installment_start_dt 0
number_of_open_accounts 0
number_of_closed_accounts 0
total_loan_months 0
delinquent_months 0
total_dpd         0
enquiry_count     0

```

```
credit_utilization_ratio      0
default                       0
dtype: int64
```

L'analyse des valeurs manquantes révèle un faible taux d'incomplétude concentré principalement sur la variable **residence_type**. Aucune autre variable ne présente de valeurs manquantes significatives.

```
1 df_train.residence_type.unique()
```

```
array(['Owned', 'Mortgage', 'Rented', nan], dtype=object)
```

```
1 mode_residence = df_train.residence_type.mode()[0]
2 mode_residence
```

```
'Owned'
```

```
1 df_train.residence_type.fillna(mode_residence, inplace=True)
2 df_test.residence_type.fillna(mode_residence, inplace=True)
3
4 df_train.residence_type.unique()
```

```
C:\Users\Hassan\AppData\Local\Temp\ipykernel_11608\3600342751.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df_train.residence_type.fillna(mode_residence, inplace=True)
C:\Users\Hassan\AppData\Local\Temp\ipykernel_11608\3600342751.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

df_test.residence_type.fillna(mode_residence, inplace=True)
]: array(['Owned', 'Mortgage', 'Rented'], dtype=object)
```

La variable **residence_type** prend 3 modalités : *Owned* (propriétaire), *Mortgage* (propriétaire avec crédit immobilier) et *Rented* (locataire).

```
1 df_train.duplicated().sum()
```

```
0
```

Le résultat indique qu'aucune observation dupliquée n'est présente dans la base d'entraînement. La qualité des données est donc satisfaisante sur ce point, et aucune opération de suppression des doublons n'est nécessaire.

```
1 df_train.describe()
```

	age	income	number_of_dependants	years_at_current_address	zipcode	sanction_amount	loan_amount	processing_fee	gst
count	37500	37500	37500	37500	37500	37500	37500	37500	37500
mean	39.54	28966.59	1.94	15.99	418866.25	51696.61	43956.75	883.20	7912.21
std	9.86	28861.59	1.54	8.92	169035.02	68836.16	59090.49	1244.41	10636.29
min	18	0	0	1	110001	0	0	0	0
25%	33	8822	0	8	302001	12562	10571	211.42	1902.78
50%	39	20735	2	16	400001	29084	24530	490.60	4415.40
75%	46	36588.75	3	24	560001	56958	50809	1016.62	9145.62
max	70	131989	5	31	700001	573925	526009	58228.98	94681.62

	net_disbursement	loan_tenure_months	principal_outstanding	bank_balance_at_application	number_of_open_accounts	number_of_closed_accounts	total_loan_months	delinquent_months	total_dpd	enquiry_count
count	37500	37500	37500	37500	37500	37500	37500	37500	37500	37500
mean	35165.40	25.96	14672.01	10849.29	2.50	1.00	76.11	4.84	26.67	5.01
std	47272.39	12.45	13350.42	11474.60	1.12	0.81	43.77	5.84	32.78	2.03
min	0	6	-0.01	0	1	0	1	0	0	1
25%	8456.80	16	4644.17	3157.44	1	0	42	0	0	4
50%	19624.00	24	10971.70	7315.16	3	1	71	3	13	5
75%	49647.20	35	19633.67	13574.69	4	2	107	8	46	6
max	420807.20	59	55000	86313.07	4	2	223	24	171	9

Les statistiques descriptives montrent des distributions globalement cohérentes pour les variables d'entrée. On note toutefois une dispersion importante sur les montants financiers (`sanction_amount`, `loan_amount`, `net_disbursement`), ce qui traduit une forte hétérogénéité des profils de crédit. Enfin, la moyenne de la variable cible (`default`) confirme un déséquilibre de classes, avec une proportion de défaut d'environ 8,6 %.

```
1 df_train.columns
```

```
Index(['cust_id', 'age', 'gender', 'marital_status', 'employment_status',
      'income', 'number_of_dependants', 'residence_type', 'years_at_current_address', 'city', 'state', 'zipcode',
      'loan_id',
      'loan_purpose', 'loan_type', 'sanction_amount', 'loan_amount',
      'processing_fee', 'gst', 'net_disbursement', 'loan_tenure_months',
      'principal_outstanding', 'bank_balance_at_application',
      'disbursal_date', 'installment_start_dt', 'number_of_open_accounts',
      'number_of_closed_accounts', 'total_loan_months', 'delinquent_months',
      'total_dpd', 'enquiry_count', 'credit_utilization_ratio', 'default'],
      dtype='object')
```

```
1 columns_continuous = ['age', 'income', 'number_of_dependants',
2                       'years_at_current_address', 'sanction_amount', 'loan_amount', 'processing_fee', 'gst', 'net_disbursement',
3                       'loan_tenure_months', 'principal_outstanding', 'bank_balance_at_application',
                        'number_of_open_accounts',
                        'number_of_closed_accounts',
```

```

4         'total_loan_months', 'delinquent_months'
5     ',
6         'total_dpd', 'enquiry_count',
7         'credit_utilization_ratio']
8 columns_categorical = ['gender', 'marital_status',
9                        'employment_status', 'residence_type',
10                       'city', 'state', 'zipcode',
11                       'loan_purpose', 'loan_type', 'default'
12 ]

```

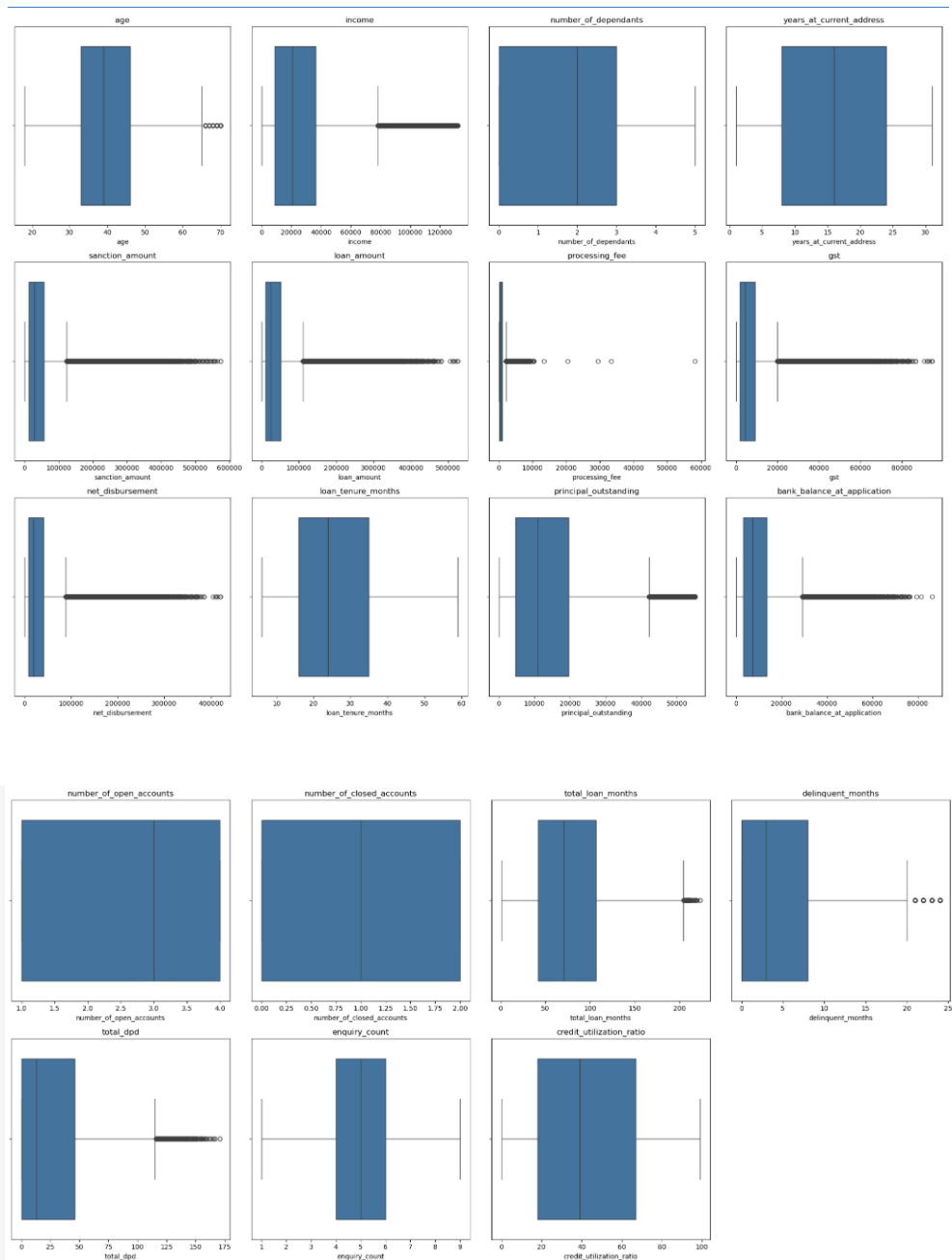
3.9 Détection et traitement des valeurs aberrantes

3.9.1 Analyse exploratoire par boxplots

```

1 num_plots = len(columns_continuous)
2 num_cols = 4
3 num_rows = (num_plots + num_cols - 1) // num_cols
4
5 fig, axes = plt.subplots(num_rows, num_cols, figsize=(5 *
6                        num_cols, 5 * num_rows))
7 axes = axes.flatten()
8
9 for i, col in enumerate(columns_continuous):
10     sns.boxplot(x=df_train[col], ax=axes[i])
11     axes[i].set_title(col)
12
13 for j in range(i + 1, num_rows * num_cols):
14     axes[j].axis('off')
15
16 plt.tight_layout()
17 plt.show()

```

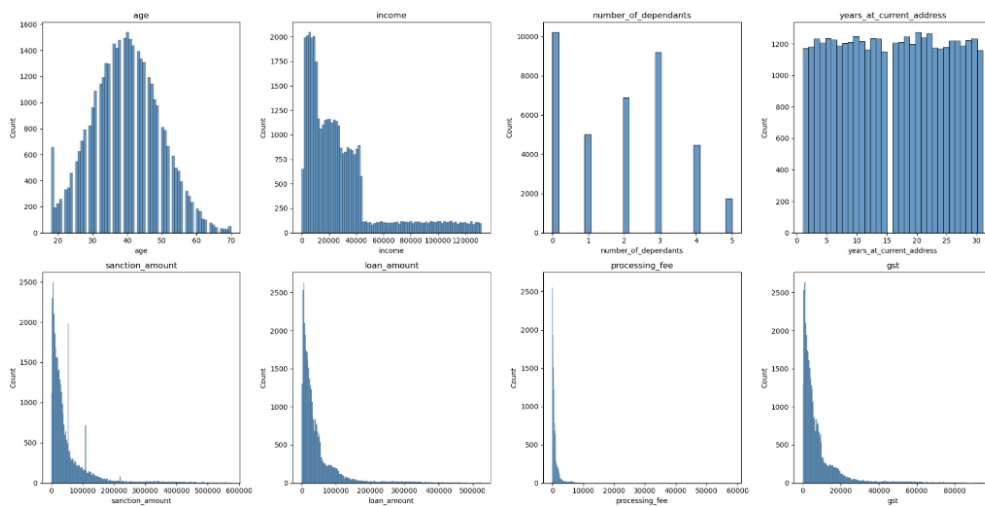


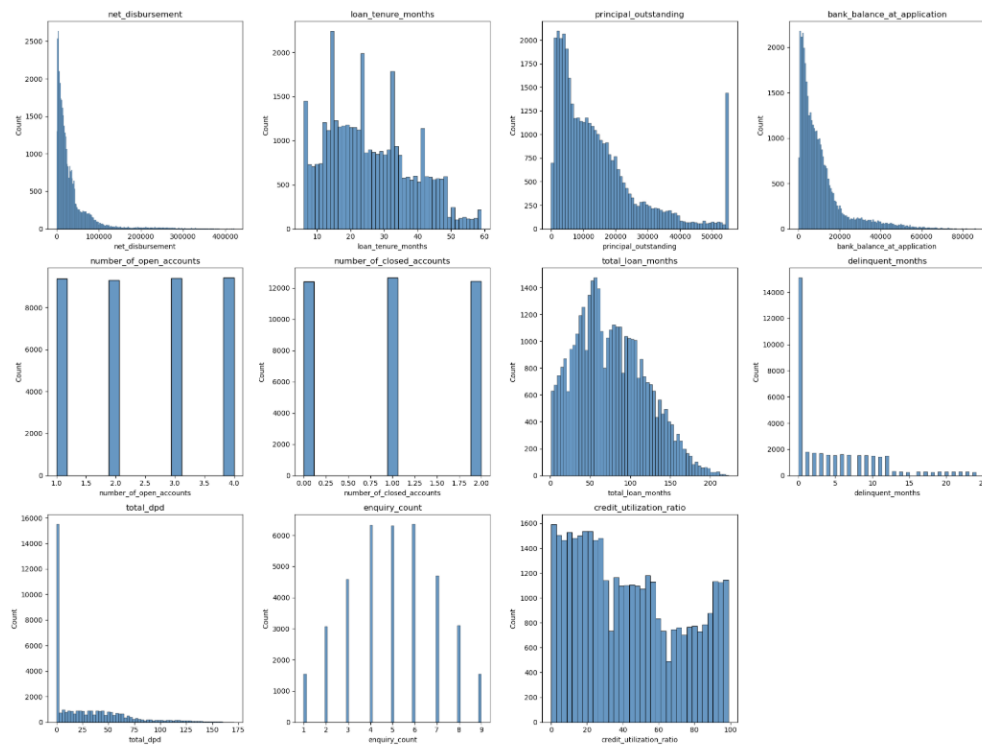
Les boxplots montrent la présence de valeurs aberrantes, principalement sur les variables monétaires (`income`, `processing_fee`, `sanction_amount`, `loan_amount`, ...), caractérisées par des distributions très étalées avec quelques valeurs extrêmement élevées.

À l'inverse, des variables comme `age` ou `delinquent_months` présentent une dispersion plus modérée et moins de valeurs extrêmes.

3.10 Analyse des distributions et asymétrie

```
1 num_plots = len(columns_continuous)
2 num_cols = 4
3 num_rows = (num_plots + num_cols - 1) // num_cols
4
5 fig, axes = plt.subplots(num_rows, num_cols, figsize=(5 *
6     num_cols, 5 * num_rows))
7 axes = axes.flatten()
8
9 for i, col in enumerate(columns_continuous):
10     sns.histplot(df_train[col], ax=axes[i])
11     axes[i].set_title(col)
12
13 for j in range(i + 1, num_rows * num_cols):
14     axes[j].axis('off')
15
16 plt.tight_layout()
17 plt.show()
```





Les histogrammes montrent que plusieurs variables monétaires (`income`, `sanction_amount`, `loan_amount`, `processing_fee`, `gst`, `net_disbursement`, `principal_outstanding`, `bank_balance_at_application`) sont fortement asymétriques à droite : la majorité des observations est concentrée sur de petites valeurs, avec une longue queue composée de valeurs très élevées.

À l'inverse, la variable `age` présente une distribution proche d'une forme en cloche, `years_at_current_address` est plus uniformément répartie, et des variables discrètes comme `number_of_dependants` et `loan_tenure_months` se concentrent sur un nombre limité de catégories.

3.10.1 Suppression des valeurs manquantes : Processing Fee

```
1 df_train.processing_fee.describe()
```

```
count    37500.000000
mean      883.197442
std       1244.409539
min        0.000000
25%       211.420000
50%       490.600000
75%      1016.620000
```



```
max      58228.978766
Name: processing_fee, dtype: float64
```

Les statistiques mettent en évidence une distribution fortement asymétrique des frais de traitement, avec une majorité de valeurs inférieures à 1 000 et quelques observations extrêmement élevées.

Ces valeurs aberrantes (pouvant atteindre 58 228,98) influencent fortement la moyenne et justifient un traitement préalable des données avant toute phase d'analyse ou de modélisation.

```
1 df_train[(df_train.processing_fee / df_train.loan_amount) >
0.03][["loan_amount", "processing_fee"]]
```

	loan_amount	processing_fee
23981	24574.0	29367.701254
28174	10626.0	13359.419404
47089	19118.0	20448.612447
29305	28776.0	33400.158058
9898	39886.0	58228.978766

Ces observations mettent en évidence des situations où les frais de traitement sont supérieurs au montant du prêt, ce qui est incohérent d'un point de vue métier et économiquement non justifiable.

Elles correspondent à des valeurs aberrantes qui doivent être corrigées ou exclues lors de la phase de nettoyage des données afin de garantir la fiabilité des analyses et des modèles prédictifs.

```
1 df_train_1 = df_train[
2     df_train.processing_fee / df_train.loan_amount < 0.03].
   copy()
3
4 df_train_1.shape
```

```
(37488, 33)
```

```
1 df_test.residence_type.isna().sum()
```

```
0
```

Le résultat confirme qu'aucune valeur manquante n'est présente dans la variable `residence_type` au sein de l'échantillon de test. Le traitement appliqué précédemment a donc permis d'assurer la cohérence entre les bases d'entraînement et de test.

```
1 df_test = df_test[
2     df_test.processing_fee / df_test.loan_amount < 0.03].copy()
   ()
```

```
3  
4 df_test.shape
```

```
(12497, 33)
```

3.10.2 Utilisation d'autres règles de gestion pour la validation des données

Règle 1. La *GST* ne doit pas être supérieure à 20 %.

```
1 df_train_1[  
2     (df_train_1.gst / df_train_1.loan_amount) > 0.2  
3 ].shape
```

```
(0, 33)
```

Le résultat indique qu'aucune observation ne viole la règle de gestion. La condition $GST / loan_amount > 20\%$ n'est vérifiée pour aucun enregistrement, ce qui confirme la cohérence des données sur ce point.

Règle 2. Le décaissement net (*net_disbursement*) ne doit pas être supérieur au montant du prêt (*loan_amount*).

```
1 df_train_1[  
2     df_train_1.net_disbursement > df_train_1.loan_amount  
3 ].shape
```

```
(0, 33)
```

Aucune observation ne viole la règle de gestion. Le montant du décaissement net (*net_disbursement*) est toujours inférieur ou égal au montant du prêt (*loan_amount*), ce qui confirme la cohérence des données sur ce point.

3.11 Analyse et correction des variables catégorielles

L'exploration des variables qualitatives a permis d'identifier une erreur typographique dans la variable *loan_purpose* ("Personaal" au lieu de "Personal"), qui a été corrigée afin d'éviter la création artificielle de modalités supplémentaires.

```
1 columns_categorical
```

```
[ 'gender',
  'marital_status',
  'employment_status',
  'residence_type',
  'city',
  'state',
  'zipcode',
  'loan_purpose',
  'loan_type',
  'default']
```

```
1 for col in columns_categorical:
2     print(col, "-->", df_train_1[col].unique())
```

```
gender --> ['M' 'F']
marital_status --> ['Married' 'Single']
employment_status --> ['Self-Employed' 'Salaried']
residence_type --> ['Owned' 'Mortgage' 'Rented']
city --> ['Hyderabad' 'Mumbai' 'Chennai' 'Bangalore' 'Pune' '
Kolkata' 'Ahmedabad'
'Delhi' 'Lucknow' 'Jaipur']
state --> ['Telangana' 'Maharashtra' 'Tamil Nadu' 'Karnataka'
'West Bengal'
'Gujarat' 'Delhi' 'Uttar Pradesh' 'Rajasthan']
zipcode --> [500001 400001 600001 560001 411001 700001 380001
110001 226001 302001]
loan_purpose --> ['Home' 'Education' 'Personal' 'Auto' '
Personaal']
loan_type --> ['Secured' 'Unsecured']
default --> [0 1]
```

3.11.1 Gestion des erreurs dans la colonne loan_purpose

```
1 df_train_1['loan_purpose'] = df_train_1['loan_purpose']\
2     .replace('Personaal', 'Personal')
3 df_train_1['loan_purpose'].unique()
```

```
array(['Home', 'Education', 'Personal', 'Auto'], dtype=object
)
```

```
1 df_test['loan_purpose'] = df_test['loan_purpose']\
2     .replace('Personaal', 'Personal')
3
4 df_test['loan_purpose'].unique()
```

```
array(['Home', 'Education', 'Auto', 'Personal'], dtype=object)
```

3.12 Analyse exploratoire des variables numériques

```
1 columns_continuous
```

```
['age',
 'income',
 'number_of_dependants',
 'years_at_current_address',
 'sanction_amount',
 'loan_amount',
 'processing_fee',
 'gst',
 'net_disbursement',
 'loan_tenure_months',
 'principal_outstanding',
 'bank_balance_at_application',
 'number_of_open_accounts',
 'number_of_closed_accounts',
 'total_loan_months',
 'delinquent_months',
 'total_dpd',
 'enquiry_count',
 'credit_utilization_ratio']
```

3.12.1 Colonne age

```
1 df_train_1.groupby("default")["age"].describe()
```

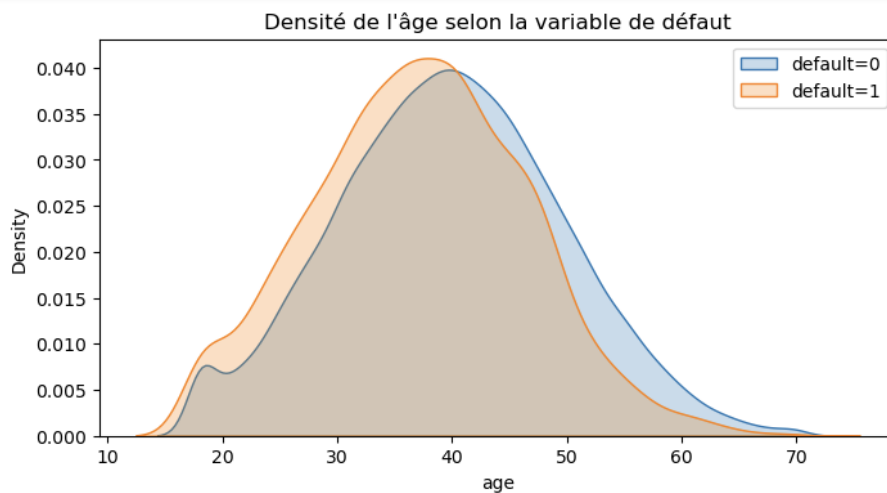
default	count	mean	std	min	25%	50%	75%	max
0	34265.000	39.768	9.880	18.000	33.000	40.000	46.000	70.000
1	3223.000	37.125	9.290	18.000	31.000	37.000	44.000	70.000

TABLE 2 – Statistiques descriptives de l'âge selon la variable cible

Les clients en défaut présentent en moyenne un âge plus faible que ceux n'ayant pas connu de défaut, bien que les distributions d'âge se chevauchent fortement.

L'âge peut ainsi constituer un facteur explicatif du risque de défaut, mais pris isolément, il demeure insuffisant pour assurer une prédiction fiable.

```
1 plt.figure(figsize=(8, 4))
2
3 sns.kdeplot(df_train_1['age'][df_train_1['default'] == 0],
4             fill=True, label='default=0')
5 sns.kdeplot(df_train_1['age'][df_train_1['default'] == 1],
6             fill=True, label='default=1')
7
8 plt.title("Densité de l'âge selon la variable de défaut")
9 plt.legend()
10 plt.show()
```



Les défauts apparaissent légèrement plus fréquents chez les clients plus jeunes, bien que l'écart observé demeure relativement modéré.

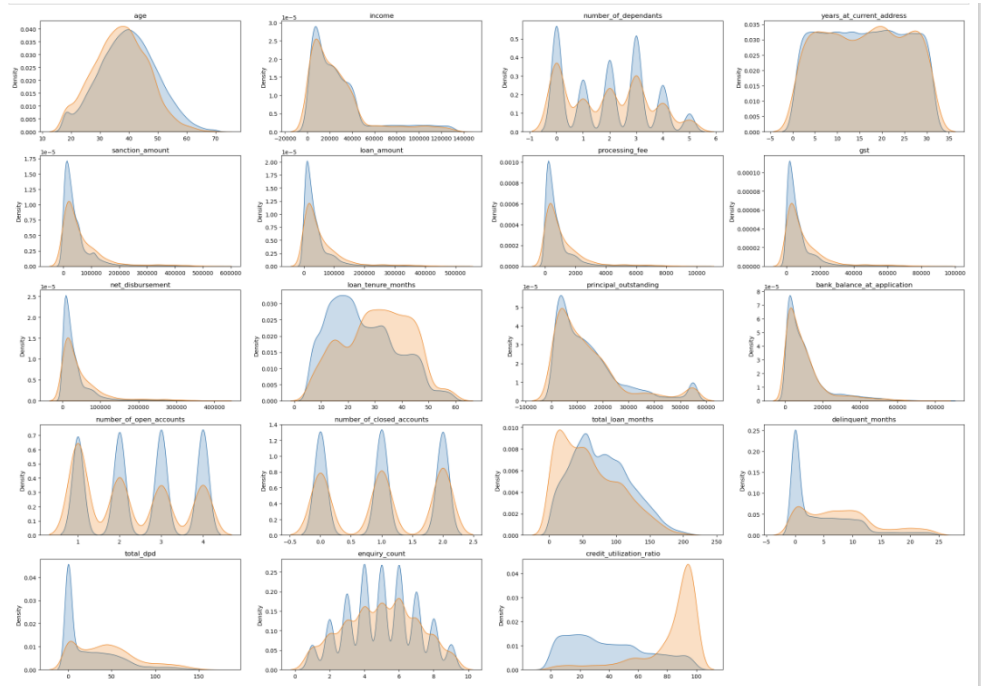
3.12.2 Analyse par densité (KDE) pour l'ensemble des variables numériques

```
1 plt.figure(figsize=(24, 20)) # Width, height in inches
2
3 for i, col in enumerate(columns_continuous):
4     plt.subplot(6, 4, i+1) # 6 rows, 4 columns, ith subplot
5     sns.kdeplot(df_train_1[col][df_train_1['default'] == 0],
6                 fill=True, label='default=0')
7     sns.kdeplot(df_train_1[col][df_train_1['default'] == 1],
8                 fill=True, label='default=1')
```

```

9     plt.title(col)
10    plt.xlabel('')
11
12    plt.tight_layout()
13    plt.show()

```



Les variables `loan_tenure_months`, `delinquent_months`, `total_dpd` et `credit_utilization_ratio` montrent qu'à mesure que leurs valeurs augmentent, la probabilité de défaut de paiement semble également croître. Ces variables apparaissent ainsi comme de bons prédicteurs du risque de défaut, dans la mesure où elles reflètent directement le comportement de remboursement et le niveau d'endettement du client.

En revanche, pour les autres variables, les distributions observées ne mettent pas en évidence de différence marquée entre les clients en défaut et les clients non défaillants. Prises individuellement, elles ne semblent donc pas présenter un pouvoir discriminant important.

Concernant `loan_amount` (montant du prêt) et `income` (revenu), aucune séparation claire n'apparaît entre les deux groupes. Cela peut s'expliquer par le fait que ces variables, considérées isolément, ne traduisent pas réellement la capacité de remboursement. Par exemple, un montant de prêt élevé n'est pas nécessairement risqué si le revenu est également élevé.

Il serait donc pertinent de créer une nouvelle variable combinant ces deux informations, telle que le ratio *Loan-to-Income* (*LTI*).

3.13 Ingénierie des caractéristiques

3.13.1 Ratio Loan-to-Income (LTI)

Définition du ratio LTI. Le ratio *Loan-to-Income* (*LTI*) est défini comme suit :

$$LTI = \frac{\text{loan_amount}}{\text{income}}$$

Les analyses montrent qu'un LTI élevé est associé à une probabilité de défaut plus importante. Cette variable constitue ainsi un indicateur robuste de la capacité de remboursement relative de l'emprunteur.

```
1 df_train_1[["loan_amount", "income"]].head(3)
```

	loan_amount	income
12746	257862.000	124597.000
32495	12639.000	7865.000
43675	124256.000	35145.000

```
1 df_train_1['loan_to_income'] = round(  
2     df_train_1['loan_amount'] / df_train_1['income'], 2)  
3 df_train_1['loan_to_income'].describe()
```

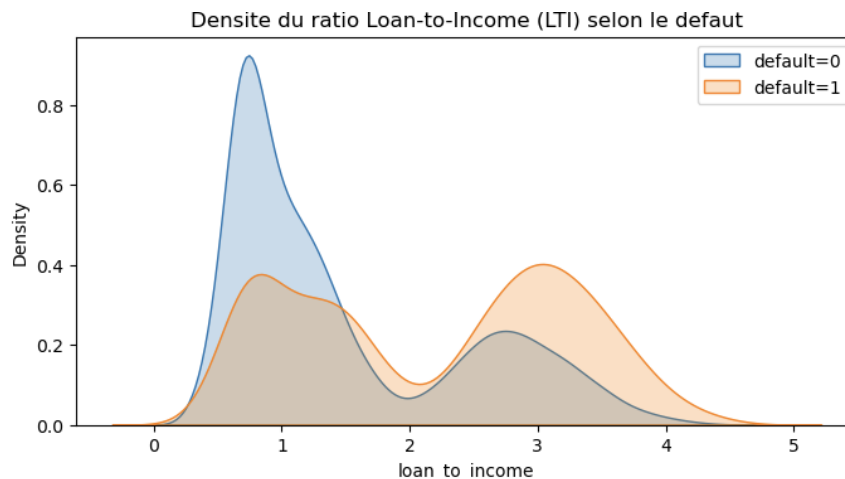
```
count      37488.000  
mean         1.557  
std          0.974  
min           0.300  
25%           0.770  
50%           1.160  
75%           2.460  
max           4.570  
Name: loan_to_income, dtype: float64
```

```
1 plt.figure(figsize=(8, 4))  
2  
3 sns.kdeplot(df_train_1['loan_to_income']  
4             [df_train_1['default'] == 0],  
5             fill=True, label='default=0')  
6  
7 sns.kdeplot(df_train_1['loan_to_income']  
8             [df_train_1['default'] == 1],  
9             fill=True, label='default=1')
```

```

10
11 plt.title("Densite du ratio Loan-to-Income (LTI) selon le
12         default")
13 plt.legend()
14 plt.show()

```



Le graphique met en évidence que les clients sans défaut (courbe bleue) présentent généralement un ratio prêt/revenu (*Loan-to-Income Ratio*, *LTI*) faible, traduisant un niveau d'endettement modéré au regard de leur revenu.

À l'inverse, les clients en défaut (courbe orange) affichent plus fréquemment un LTI élevé, ce qui suggère que plus le montant du prêt représente une part importante du revenu, plus le risque de défaut augmente.

Ainsi, un ratio prêt/revenu élevé constitue un indicateur pertinent de prêt à haut risque.

3.13.2 Taux de délinquance (impayés)

Définition du taux de délinquance. Le *Delinquency Ratio* est défini comme suit :

$$\text{Delinquency Ratio} = \frac{\text{delinquent_months}}{\text{total_loan_months}}$$

Ce ratio mesure la proportion du temps de crédit durant lequel le client a été en situation de retard de paiement.

```

1 df_train_1['delinquency_ratio'] = (
2     df_train_1['delinquent_months'] * 100
3     / df_train_1['total_loan_months']
4 ).round(1)

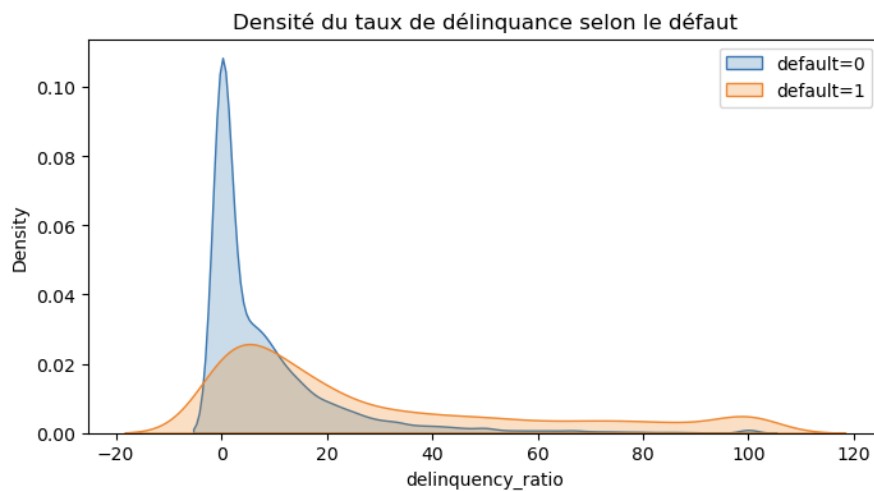
```



```

5 df_test['delinquency_ratio'] = (
6     df_test['delinquent_months'] * 100
7     / df_test['total_loan_months']
8 ) .round(1)
9
10
11 plt.figure(figsize=(8, 4))
12
13 sns.kdeplot(df_train_1['delinquency_ratio']
14             [df_train_1['default'] == 0],
15             fill=True, label='default=0')
16
17 sns.kdeplot(df_train_1['delinquency_ratio']
18             [df_train_1['default'] == 1],
19             fill=True, label='default=1')
20
21 plt.title("Densité du taux de délinquance selon le défaut")
22 plt.legend()
23 plt.show()

```



La courbe bleue (`default = 0`) met en évidence une forte concentration des observations dans la partie basse du taux de délinquance (PSR), ce qui indique que la majorité des clients non défaillants présentent un faible niveau de retard de paiement.

À l'inverse, la courbe orange (`default = 1`) présente une densité plus marquée pour des niveaux élevés du PSR, suggérant une relation positive entre l'augmentation du taux de délinquance et le risque de défaut.

3.13.3 Gravité moyenne des retards

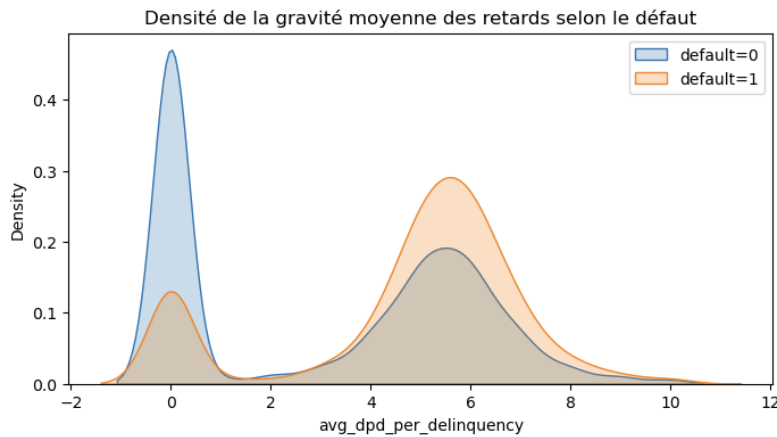
Définition de l’Average DPD. L’*Average DPD* est défini comme suit :

$$\text{Average DPD} = \frac{\text{total_dpd}}{\text{delinquent_months}}$$

Cette variable mesure l’intensité moyenne des retards de paiement et s’est révélée fortement discriminante dans l’analyse du risque de défaut.

```
1 df_train_1['avg_dpd_per_delinquency'] = np.where(df_train_1['delinquent_months'] != 0, (df_train_1['total_dpd'] / df_train_1['delinquent_months']).round(1), 0)
2
3 df_test['avg_dpd_per_delinquency'] = np.where(
4     df_test['delinquent_months'] != 0,
5     (df_test['total_dpd']
6      / df_test['delinquent_months']).round(1),
7     0
8 )
```

```
1 plt.figure(figsize=(8, 4))
2
3 sns.kdeplot(df_train_1['avg_dpd_per_delinquency']
4             [df_train_1['default'] == 0],
5             fill=True, label='default=0')
6
7 sns.kdeplot(df_train_1['avg_dpd_per_delinquency']
8             [df_train_1['default'] == 1],
9             fill=True, label='default=1')
10
11 plt.title("Densite de la gravite moyenne des retards selon le
12           default")
13 plt.legend()
14 plt.show()
```



La courbe bleue (`default = 0`) met en évidence une forte concentration des valeurs autour de 0, ce qui indique que les clients non défaillants présentent peu ou pas de retard moyen.

À l'inverse, la courbe orange (`default = 1`) est décalée vers des valeurs plus élevées, traduisant des retards plus longs et plus sévères.

Le faible chevauchement entre les deux distributions suggère que la variable `avg_dpd_per_delinquency` constitue un indicateur fortement discriminant du risque de défaillance.

3.14 Suppression des variables redondantes

```
1 df_train_1.columns
```

```
Index(['cust_id', 'age', 'gender', 'marital_status', 'employment_status',
      'income', 'number_of_dependants', 'residence_type',
      'years_at_current_address', 'city', 'state', 'zipcode',
      'loan_id',
      'loan_purpose', 'loan_type', 'sanction_amount', 'loan_amount',
      'processing_fee', 'gst', 'net_disbursement', 'loan_tenure_months',
      'principal_outstanding', 'bank_balance_at_application',
      'disbursal_date', 'installment_start_dt', 'number_of_open_accounts',
      'number_of_closed_accounts', 'total_loan_months', 'delinquent_months',
      'total_dpd', 'enquiry_count', 'credit_utilization_ratio', 'default',
```

```

        'loan_to_income', 'delinquency_ratio', '
        avg_dpd_per_delinquency'],
        dtype='object')

```

```

1 df_train_2 = df_train_1.drop(['cust_id', 'loan_id'], axis="
    columns")
2 df_test = df_test.drop(['cust_id', 'loan_id'], axis="columns"
    )

```

```

1 df_train_3 = df_train_2.drop(
2     ['disbursal_date', 'installment_start_dt',
3     'loan_amount', 'income',
4     'total_loan_months', 'delinquent_months',
5     'total_dpd'],
6     axis="columns"
7 )
8
9 df_test = df_test.drop(
10    ['disbursal_date', 'installment_start_dt',
11    'loan_amount', 'income',
12    'total_loan_months', 'delinquent_months',
13    'total_dpd'],
14    axis="columns"
15 )
16
17 df_train_3.columns

```

```

Index(['age', 'gender', 'marital_status', 'employment_status'
,
    'number_of_dependants', 'residence_type',
    'years_at_current_address', 'city', 'state', 'zipcode'
,
    'loan_purpose', 'loan_type', 'sanction_amount',
    'processing_fee', 'gst', 'net_disbursement',
    'loan_tenure_months', 'principal_outstanding',
    'bank_balance_at_application', '
number_of_open_accounts',
    'number_of_closed_accounts', 'enquiry_count',
    'credit_utilization_ratio', 'default', 'loan_to_income'
,
    'delinquency_ratio', 'avg_dpd_per_delinquency'],
dtype='object')

```

```

1 df_train_3.select_dtypes(['int64', 'float64']).columns

```

```

Index(['age', 'number_of_dependants', '
years_at_current_address', 'zipcode',

```

```

        'sanction_amount', 'processing_fee', 'gst', '
net_disbursement',
        'loan_tenure_months', 'principal_outstanding',
        'bank_balance_at_application', '
number_of_open_accounts',
        'number_of_closed_accounts', 'enquiry_count',
        'credit_utilization_ratio', 'loan_to_income',
        'delinquency_ratio', 'avg_dpd_per_delinquency'],
dtype='object')

```

Les variables identifiants (`cust_id`, `loan_id`), ainsi que certaines variables fortement corrélées ou redondantes, ont été supprimées afin d'éviter toute information non pertinente ou dupliquée dans le modèle.

3.15 Analyse de la multicollinéarité (VIF)

Le *Variance Inflation Factor (VIF)* a été utilisé afin d'évaluer le niveau de colinéarité entre les variables numériques.

Les résultats initiaux ont mis en évidence des valeurs infinies pour certaines variables parfaitement corrélées. Après suppression des variables redondantes, les VIF ont été ramenés à des niveaux jugés acceptables (inférieurs à 10 pour la majorité des variables), indiquant une réduction significative des problèmes de multicollinéarité.

```

1 from sklearn.preprocessing import MinMaxScaler
2
3 X_train = df_train_3.drop('default', axis='columns')
4 y_train = df_train_3['default']
5
6 cols_to_scale = X_train.select_dtypes(['int64', 'float64']).
   columns
7
8 scaler = MinMaxScaler()
9
10 X_train[cols_to_scale] = scaler.fit_transform(
11     X_train[cols_to_scale]
12 )
13
14 X_train.describe()

```

[569]:

	age	number_of_dependants	years_at_current_address	zipcode	sanction_amount	processing_fee	gst	net_disbursement	loan_tenure_months
count	37488.000	37488.000	37488.000	37488.000	37488.000	37488.000	37488.000	37488.000	37488.000
mean	0.414	0.389	0.500	0.524	0.089	0.083	0.083	0.083	0.377
std	0.190	0.307	0.297	0.286	0.120	0.112	0.112	0.112	0.235
min	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
25%	0.288	0.000	0.233	0.325	0.021	0.019	0.019	0.019	0.189
50%	0.404	0.400	0.500	0.492	0.049	0.046	0.046	0.046	0.340
75%	0.538	0.600	0.767	0.763	0.098	0.096	0.096	0.096	0.547
max	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

number_of_open_accounts	number_of_closed_accounts	enquiry_count	credit_utilization_ratio	loan_to_income	delinquency_ratio	avg_dpd_per_delinquency
37488.000	37488.000	37488.000	37488.000	37488.000	37488.000	37488.000
0.501	0.501	0.501	0.439	0.294	0.103	0.328
0.373	0.407	0.254	0.297	0.228	0.173	0.291
0.000	0.000	0.000	0.000	0.000	0.000	0.000
0.000	0.000	0.375	0.182	0.110	0.000	0.000
0.667	0.500	0.500	0.394	0.201	0.037	0.430
1.000	1.000	0.625	0.677	0.506	0.129	0.573
1.000	1.000	1.000	1.000	1.000	1.000	1.000

3.16 Même transformation sur les ensembles de test

```

1 X_test = df_test.drop('default', axis='columns')
2 y_test = df_test['default']
3
4 X_test[cols_to_scale] = scaler.transform(
5     X_test[cols_to_scale]
6 )
7
8 X_test.describe()

```

	age	number_of_dependants	years_at_current_address	zipcode	sanction_amount	processing_fee	gst	net_disbursement	loan_tenure_months	pi
count	12497.000	12497.000	12497.000	12497.000	12497.000	12497.000	12497.000	12497.000	12497.000	12497.000
mean	0.415	0.385	0.503	0.525	0.089	0.083	0.083	0.083	0.375	
std	0.189	0.307	0.298	0.286	0.121	0.113	0.113	0.113	0.234	
min	0.000	0.000	0.000	0.000	-0.000	-0.000	-0.000	-0.000	0.000	
25%	0.288	0.000	0.233	0.325	0.021	0.019	0.019	0.019	0.189	
50%	0.423	0.400	0.500	0.510	0.050	0.046	0.046	0.046	0.340	
75%	0.538	0.600	0.767	0.763	0.097	0.095	0.095	0.095	0.547	
max	1.000	1.000	1.000	1.000	0.981	0.964	0.964	0.964	1.000	

number_of_open_accounts	number_of_closed_accounts	enquiry_count	credit_utilization_ratio	loan_to_income	delinquency_ratio	avg_dpd_per_delinquency
12497.000	12497.000	12497.000	12497.000	12497.000	12497.000	12497.000
0.497	0.500	0.501	0.436	0.293	0.106	0.334
0.373	0.407	0.254	0.295	0.226	0.173	0.290
0.000	0.000	0.000	0.000	0.000	0.000	0.000
0.000	0.000	0.375	0.182	0.112	0.000	0.000
0.333	0.500	0.500	0.394	0.201	0.042	0.440
0.667	1.000	0.625	0.677	0.499	0.134	0.580
1.000	1.000	1.000	1.000	1.005	1.000	1.000

```

1 from statsmodels.stats.outliers_influence import
  variance_inflation_factor
2
3 def calculate_vif(data):
4     vif_df = pd.DataFrame()
5     vif_df['Column'] = data.columns
6     vif_df['VIF'] = [
7         variance_inflation_factor(data.values, i)
8         for i in range(data.shape[1])
9     ]
10    return vif_df

```

```

1 X_train.head(4)

```

[573]:

	age	gender	marital_status	employment_status	number_of_dependants	residence_type	years_at_current_address	city	state	zipcode
12746	0.788	M	Married	Self-Employed	0.600	Owned	0.967	Hyderabad	Telangana	0.661
32495	0.500	F	Single	Salaried	0.000	Owned	0.867	Mumbai	Maharashtra	0.492
43675	0.385	M	Single	Salaried	0.000	Mortgage	0.833	Chennai	Tamil Nadu	0.831
9040	0.462	M	Married	Salaried	0.400	Mortgage	0.967	Bangalore	Karnataka	0.763

[573]:

	loan_purpose	loan_type	sanction_amount	processing_fee	gst	net_disbursement	loan_tenure_months	principal_outstanding	bank_balance_at_application
	Home	Secured	0.634	0.490	0.490	0.490	0.415	1.000	0.613
	Education	Secured	0.021	0.023	0.023	0.023	0.830	0.087	0.025
	Home	Secured	0.218	0.235	0.235	0.235	0.491	0.327	0.174
	Education	Secured	0.043	0.034	0.034	0.034	0.642	0.199	0.078

number_of_open_accounts	number_of_closed_accounts	enquiry_count	credit_utilization_ratio	loan_to_income	delinquency_ratio	avg_dpd_per_delinquency
1.000	1.000	0.375	0.364	0.415	0.132	0.590
0.667	0.500	0.500	0.051	0.307	0.062	0.620
0.000	0.500	0.375	0.000	0.759	0.222	0.560
0.667	0.000	0.875	0.879	0.194	0.000	0.000

```

1 calculate_vif(X_train[cols_to_scale])

```

```
C:\Users\Hassan\anaconda3\Lib\site-packages\statsmodels\
stats\outliers_influence.py:197: RuntimeWarning:
divide by zero encountered in scalar divide
vif = 1. / (1. - r_squared_i)
```

Ce message d'avertissement indique la présence d'une corrélation parfaite entre certaines variables explicatives, ce qui entraîne une division par zéro dans le calcul du VIF. Cela traduit un problème de multicollinéarité sévère, nécessitant la suppression ou la transformation des variables concernées.

Index	Column	VIF
0	age	5.701
1	number_of_dependants	2.730
2	years_at_current_address	3.423
3	zipcode	3.798
4	sanction_amount	101.087
5	processing_fee	inf
6	gst	inf
7	net_disbursement	inf
8	loan_tenure_months	6.181
9	principal_outstanding	16.326
10	bank_balance_at_application	9.342
11	number_of_open_accounts	4.386
12	number_of_closed_accounts	2.385
13	enquiry_count	6.410
14	credit_utilization_ratio	2.933
15	loan_to_income	6.962
16	delinquency_ratio	1.935
17	avg_dpd_per_delinquency	2.909

TABLE 3 – Variance Inflation Factor (VIF) des variables numériques

Les résultats du *Variance Inflation Factor (VIF)* mettent en évidence une multicollinéarité sévère, en particulier entre les variables financières liées au montant du prêt, dont certaines présentent un VIF infini.

Cette situation traduit l'existence de relations linéaires quasi parfaites entre certaines variables explicatives. Il est donc nécessaire de supprimer les variables redondantes afin d'améliorer la stabilité et l'interprétabilité du modèle.


```
1 corr = X_train[cols_to_scale].corr().abs()
2
3 corr
```

	1991	1992	1993	1994	1995	1996	1997	1998	1999	2000	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	2027	2028	2029	2030	2031	2032	2033	2034	2035	2036	2037	2038	2039	2040	2041	2042	2043	2044	2045	2046	2047	2048	2049	2050	2051	2052	2053	2054	2055	2056	2057	2058	2059	2060	2061	2062	2063	2064	2065	2066	2067	2068	2069	2070	2071	2072	2073	2074	2075	2076	2077	2078	2079	2080	2081	2082	2083	2084	2085	2086	2087	2088	2089	2090	2091	2092	2093	2094	2095	2096	2097	2098	2099	2100	2101	2102	2103	2104	2105	2106	2107	2108	2109	2110	2111	2112	2113	2114	2115	2116	2117	2118	2119	2120	2121	2122	2123	2124	2125	2126	2127	2128	2129	2130	2131	2132	2133	2134	2135	2136	2137	2138	2139	2140	2141	2142	2143	2144	2145	2146	2147	2148	2149	2150	2151	2152	2153	2154	2155	2156	2157	2158	2159	2160	2161	2162	2163	2164	2165	2166	2167	2168	2169	2170	2171	2172	2173	2174	2175	2176	2177	2178	2179	2180	2181	2182	2183	2184	2185	2186	2187	2188	2189	2190	2191	2192	2193	2194	2195	2196	2197	2198	2199	2200	2201	2202	2203	2204	2205	2206	2207	2208	2209	2210	2211	2212	2213	2214	2215	2216	2217	2218	2219	2220	2221	2222	2223	2224	2225	2226	2227	2228	2229	2230	2231	2232	2233	2234	2235	2236	2237	2238	2239	2240	2241	2242	2243	2244	2245	2246	2247	2248	2249	2250	2251	2252	2253	2254	2255	2256	2257	2258	2259	2260	2261	2262	2263	2264	2265	2266	2267	2268	2269	2270	2271	2272	2273	2274	2275	2276	2277	2278	2279	2280	2281	2282	2283	2284	2285	2286	2287	2288	2289	2290	2291	2292	2293	2294	2295	2296	2297	2298	2299	2300	2301	2302	2303	2304	2305	2306	2307	2308	2309	2310	2311	2312	2313	2314	2315	2316	2317	2318	2319	2320	2321	2322	2323	2324	2325	2326	2327	2328	2329	2330	2331	2332	2333	2334	2335	2336	2337	2338	2339	2340	2341	2342	2343	2344	2345	2346	2347	2348	2349	2350	2351	2352	2353	2354	2355	2356	2357	2358	2359	2360	2361	2362	2363	2364	2365	2366	2367	2368	2369	2370	2371	2372	2373	2374	2375	2376	2377	2378	2379	2380	2381	2382	2383	2384	2385	2386	2387	2388	2389	2390	2391	2392	2393	2394	2395	2396	2397	2398	2399	2400	2401	2402	2403	2404	2405	2406	2407	2408	2409	2410	2411	2412	2413	2414	2415	2416	2417	2418	2419	2420	2421	2422	2423	2424	2425	2426	2427	2428	2429	2430	2431	2432	2433	2434	2435	2436	2437	2438	2439	2440	2441	2442	2443	2
--	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	---

TABLE 4 – Matrice de corrélation des variables numériques

```
1 high_corr = corr[corr > 0.95]
2
3 high_corr
```

	sanction_amount	processing_fee	gst	net_disbursement
sanction_amount	1.000	0.992	0.992	0.992
processing_fee	0.992	1.000	1.000	1.000
gst	0.992	1.000	1.000	1.000
net_disbursement	0.992	1.000	1.000	1.000

TABLE 5 – Corrélations supérieures à 0.95 entre variables financières

On observe une forte corrélation entre les variables `sanction_amount`, `processing_fee`, `gst`, `net_disbursement` et `principal_outstanding`.

Ces variables étant toutes liées au montant du prêt, elles contiennent une information largement redondante. Leur coexistence dans le modèle peut engendrer un problème de multicollinéarité, susceptible d'affecter la stabilité des coefficients et l'interprétabilité du modèle.

```

1 features_to_drop_vif = [
2     'sanction_amount',
3     'processing_fee',
4     'gst',
5     'net_disbursement',
6     'principal_outstanding'
7 ]
8
9 X_train_1 = X_train.drop(features_to_drop_vif, axis='columns'
10 )
11
12 numeric_columns = X_train_1.select_dtypes(['int64', 'float64',
13 ]).columns

```

```
Index(['age', 'number_of_dependants', 'years_at_current_address', 'zipcode', 'loan_tenure_months', 'bank_balance_at_application', 'number_of_open_accounts', 'number_of_closed_accounts', 'enquiry_count', 'credit_utilization_ratio', 'loan_to_income', 'delinquency_ratio', 'avg_dpd_per_delinquency'], dtype='object')
```

```
1 calculate_vif(X_train_1[numeric_columns])
```

Index	Column	VIF
0	age	5.429
1	number_of_dependants	2.727
2	years_at_current_address	3.404
3	zipcode	3.778
4	loan_tenure_months	6.019
5	bank_balance_at_application	1.805
6	number_of_open_accounts	4.353
7	number_of_closed_accounts	2.372
8	enquiry_count	6.384
9	credit_utilization_ratio	2.920
10	loan_to_income	4.561
11	delinquency_ratio	1.934
12	avg_dpd_per_delinquency	2.906

TABLE 6 – Variance Inflation Factor (VIF) après suppression des variables redondantes

Les valeurs du *Variance Inflation Factor (VIF)* sont globalement inférieures à 10, ce qui indique l'absence de multicolinéarité sévère dans le modèle.

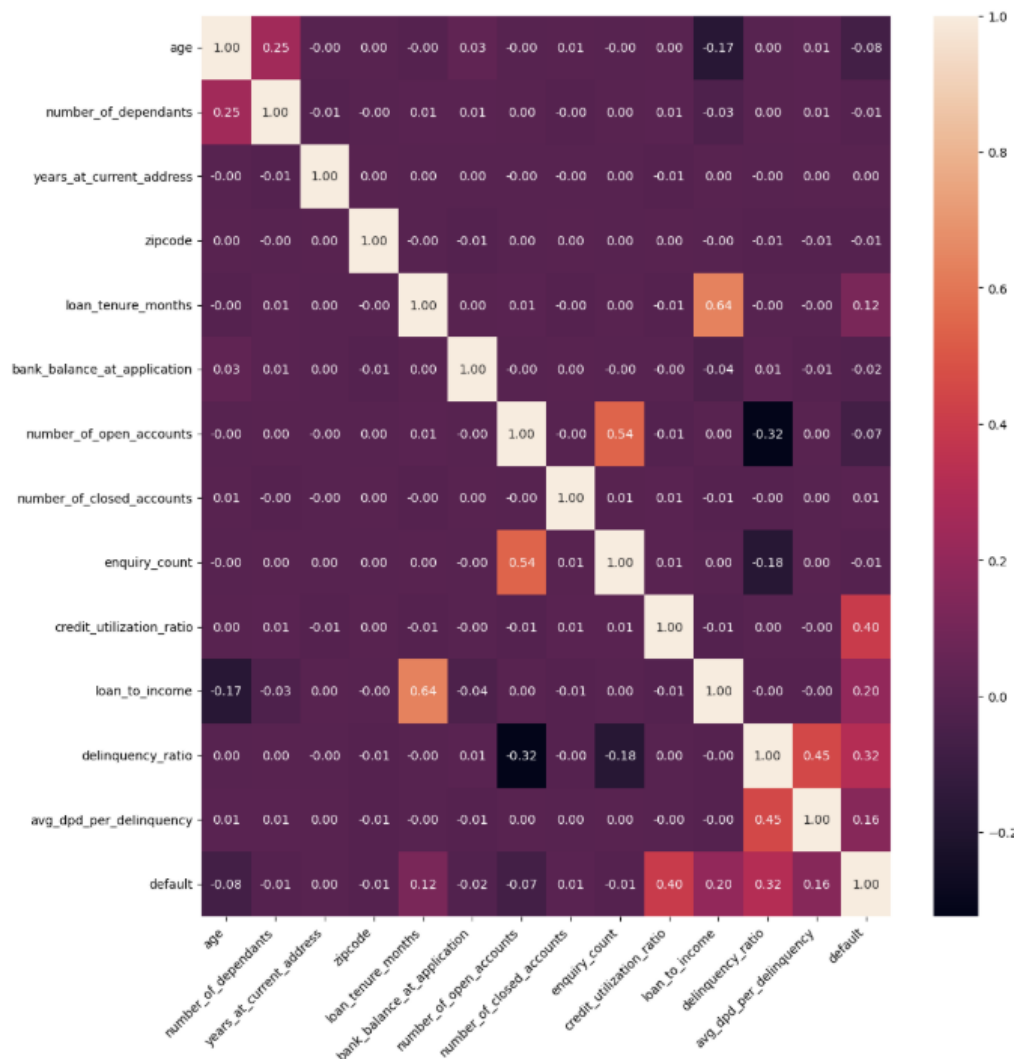
Certaines variables présentent néanmoins une corrélation modérée (VIF compris entre 5 et 7), mais ce niveau reste acceptable et ne compromet pas la stabilité des estimations.

```
1 selected_numeric_features_vif = vif_df.Column.values
2
3 selected_numeric_features_vif
```

```

1 plt.figure(figsize=(12,12))
2
3 cm = df_train_3[
4     numeric_columns.append(pd.Index(['default']))
5 ].corr()
6
7 sns.heatmap(cm, annot=True, fmt='0.2f')
8
9 plt.xticks(rotation=45, ha='right')
10 plt.yticks(rotation=0)
11
12 plt.tight_layout()
13 plt.show()

```



Carte thermique de la matrice de corrélation entre variables explicatives numériques et variable de défaut

3.17 Sélection des variables par Information Value (IV)

Définition du WOE et de l'Information Value

Le **Weight of Evidence (WOE)** mesure le pouvoir de séparation d'une modalité d'une variable par rapport à la variable cible.

L'**Information Value (IV)** mesure la capacité prédictive globale d'une variable.

Le *Weight of Evidence (WOE)* et l'*Information Value (IV)* ont été calculés pour chaque variable explicative.

Règle d'interprétation classique de l'IV :

- $IV < 0.02$: variable non prédictive
- $0.02 \leq IV < 0.1$: faible pouvoir prédictif
- $0.1 \leq IV < 0.3$: pouvoir prédictif moyen
- $IV \geq 0.3$: fort pouvoir prédictif

Les variables présentant un IV supérieur à 0.02 ont été retenues pour la phase de modélisation.

```
1 # Fonction pour calculer le WOE (Weight of Evidence)
2 # et l'IV (Information Value) d'une variable
3
4 def calculate_woe_iv(df, feature, target):
5
6     # 1      Regrouper les données par modalité de la
7     variable
8     # count = nombre total d'observations
9     # sum = nombre de "good"
10    grouped = df.groupby(feature)[target].agg(['count', 'sum',
11    ])
12
13    # 2      Renommer les colonnes
14    grouped = grouped.rename(columns={
15        'count': 'total',
16        'sum': 'good',
17    })
18
19    # 3      Calculer le nombre de "bad"
20    grouped['bad'] = grouped['total'] - grouped['good']
21
22    # 4      Calculer les totaux globaux
23    total_good = grouped['good'].sum()
```

```

22     total_bad = grouped['bad'].sum()
23
24     # 5         Calcul des proportions
25     grouped['good_pct'] = grouped['good'] / total_good
26     grouped['bad_pct'] = grouped['bad'] / total_bad
27
28     # 6         Calcul du WOE
29     grouped['woe'] = np.log(grouped['good_pct'] / grouped['
bad_pct'])
30
31     # 7         Calcul de l'IV par modalit  
32     grouped['iv'] = (
33         grouped['good_pct'] - grouped['bad_pct']
34     ) * grouped['woe']
35
36     # 8         Remplacer les valeurs infinies
37     grouped['woe'] = grouped['woe'].replace(
38         [np.inf, -np.inf], 0
39     )
40     grouped['iv'] = grouped['iv'].replace(
41         [np.inf, -np.inf], 0
42     )
43
44     # 9         Calcul de l'IV total
45     total_iv = grouped['iv'].sum()
46
47     # 10        Retourner les r  sultats
48     return grouped, total_iv
49
50
51 # Fusion des variables explicatives et de la cible
52 df_woe = pd.concat([X_train_1, y_train], axis=1)
53
54 # Calcul WOE et IV
55 grouped, total_iv = calculate_woe_iv(
56     df_woe, 'loan_purpose', 'default'
57 )
58
59 # Affichage des r  sultats
60 print("Information Value (IV) total :", total_iv)
61 grouped

```

```
Information Value (IV) total : 0.3691197842282755
```

	iv	total	good	bad	good_pct	bad_pct	woe
loan_purpose							
Auto	0.076	7447	327	7120	0.101	0.208	-0.717

Education	5620	559	5061	0.173	0.148	0.161
0.004						
Home	11304	1734	9570	0.538	0.279	0.656
0.170						
Personal	13117	603	12514	0.187	0.365	-0.669
0.119						

La modalité **Home** présente un WOE positif et la contribution IV la plus élevée, indiquant une forte association avec les clients non défaillants.

Les modalités **Auto** et **Personal** ont des WOE négatifs, ce qui suggère un risque de défaillance plus élevé pour ces types de prêts.

La modalité **Education** montre un effet faible et peu discriminant.

Globalement, la variable **loan_purpose** apporte une information utile pour la prédiction du défaut, principalement grâce aux modalités **Home** et **Personal**.

```
1 X_train_1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 37488 entries, 12746 to 37784
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                   37488 non-null  float64
1   gender                               37488 non-null  object
2   marital_status                       37488 non-null  object
3   employment_status                   37488 non-null  object
4   number_of_dependants                 37488 non-null  float64
5   residence_type                       37488 non-null  object
6   years_at_current_address             37488 non-null  float64
7   city                                 37488 non-null  object
8   state                                37488 non-null  object
9   zipcode                              37488 non-null  float64
10  loan_purpose                           37488 non-null  object
11  loan_type                             37488 non-null  object
12  loan_tenure_months                   37488 non-null  float64
13  bank_balance_at_application          37488 non-null  float64
14  number_of_open_accounts              37488 non-null  float64
15  number_of_closed_accounts            37488 non-null  float64
16  enquiry_count                        37488 non-null  float64
17  credit_utilization_ratio             37488 non-null  float64
18  loan_to_income                       37488 non-null  float64
19  delinquency_ratio                    37488 non-null  float64
20  avg_dpd_per_delinquency              37488 non-null  float64
dtypes: float64(13), object(8)
memory usage: 6.3+ MB
```

Ce script permet de mesurer le pouvoir prédictif (*Information Value – IV*) de chaque variable du jeu de données `X_train_1`.

Les variables catégorielles (de type `object`) sont traitées directement. Les variables numériques sont d'abord discrétisées en classes avant le calcul de l'IV.

Le résultat final est stocké dans le dictionnaire `iv_values`, associant chaque variable à son IV global.

```
1 iv_values = {}
2
3 for feature in X_train_1.columns:
4     if X_train_1[feature].dtype == 'object':
5         _, iv = calculate_woe_iv(pd.concat([X_train_1,
6         y_train], axis=1), feature, 'default')
7     else:
8         X_binned = pd.cut(
9             X_train_1[feature],
10            bins=10,
11            labels=False
12        )
13        _, iv = calculate_woe_iv(
14            pd.concat([X_binned, y_train], axis=1),
15            feature,
16            'default'
17        )
18
19     iv_values[feature] = iv
20
21 iv_values
```

```
{'age': 0.0890689462679479,
'gender': 0.00047449502170914947,
'marital_status': 0.001129766845390142,
'employment_status': 0.003953046301722585,
'number_of_dependants': 0.0019380899135053508,
'residence_type': 0.246745268718145,
'years_at_current_address': 0.0020800513608156363,
'city': 0.0019059578709781529,
'state': 0.0019005589806779287,
'zipcode': 0.0016677413243392572,
'loan_purpose': 0.3691197842282755,
'loan_type': 0.16319324904149224,
'loan_tenure_months': 0.21893515090196278,
'bank_balance_at_application': 0.0063187993277516365,
'number_of_open_accounts': 0.08463134083005877,
'number_of_closed_accounts': 0.0011964272592421567,
'enquiry_count': 0.007864214085342608,
```

```
'credit_utilization_ratio': 2.352965568168245,
'loan_to_income': 0.476415456948364,
'delinquency_ratio': 0.716576108689321,
'avg_dpd_per_delinquency': 0.40151905412190175}
```

```
1 pd.set_option('display.float_format', lambda x: '{:.3f}'.
   format(x))
2
3 iv_df = pd.DataFrame(list(iv_values.items()), columns=['
   Feature', 'IV'])
4 iv_df = iv_df.sort_values(by='IV', ascending=False)
5
6 iv_df
```

Index	Feature	IV
17	credit_utilization_ratio	2.353
19	delinquency_ratio	0.717
18	loan_to_income	0.476
20	avg_dpd_per_delinquency	0.402
10	loan_purpose	0.369
5	residence_type	0.247
12	loan_tenure_months	0.219
11	loan_type	0.163
0	age	0.089
14	number_of_open_accounts	0.085
16	enquiry_count	0.008
13	bank_balance_at_application	0.006
3	employment_status	0.004
6	years_at_current_address	0.002
4	number_of_dependants	0.002
7	city	0.002
8	state	0.002
9	zipcode	0.002
15	number_of_closed_accounts	0.001
2	marital_status	0.001
1	gender	0.000

TABLE 7 – Classement des variables selon leur Information Value (IV)

3.17.1 Sélection des variables selon l'Information Value

On sélectionne les variables explicatives dont l'*Information Value (IV)* est supérieure à 0.02.

```
1 selected_features_iv = [feature for feature, iv in iv_values.  
    items() if iv > 0.02]  
2  
3 selected_features_iv
```

```
['age',  
'residence_type',  
'loan_purpose',  
'loan_type',  
'loan_tenure_months',  
'number_of_open_accounts',  
'credit_utilization_ratio',  
'loan_to_income',  
'delinquency_ratio',  
'avg_dpd_per_delinquency']
```

4 CHAPITRE 3 : MODÉLISATION ET ÉVALUATION DES PERFORMANCES

4.1 Encodage des variables

Encodage des variables catégorielles. Les variables catégorielles ont été transformées via *One-Hot Encoding* (`drop_first=True`) afin d'éviter le piège des variables fictives.

```
1 X_train_reduced = X_train_1[selected_features_iv]  
2 X_test_reduced = X_test[selected_features_iv]
```

```
1 X_train_encoded = pd.get_dummies(X_train_reduced, drop_first=  
    True)  
2  
3 X_train_encoded.head(5)
```

```
154]:
```

	age	loan_tenure_months	number_of_open_accounts	credit_utilization_ratio	loan_to_income	delinquency_ratio	avg_dpd_per_delinquency
12746	0.788	0.415	1.000	0.364	0.415	0.132	0.590
32495	0.500	0.830	0.667	0.051	0.307	0.062	0.620
43675	0.385	0.491	0.000	0.000	0.759	0.222	0.560
9040	0.462	0.642	0.667	0.879	0.194	0.000	0.000
13077	0.769	0.170	0.000	0.717	0.047	0.000	0.000

avg_dpd_per_delinquency	residence_type_Owned	residence_type_Rented	loan_purpose_Education	loan_purpose_Home	loan_purpose_Personal	loan_type_Unsecured
0.590	True	False	False	True	False	False
0.620	True	False	True	False	False	False
0.560	False	False	False	True	False	False
0.000	False	False	True	False	False	False
0.000	True	False	False	False	True	True

4.2 Stratégie expérimentale

Stratégie d'entraînement des modèles. L'entraînement des modèles a été structuré en trois tentatives successives afin d'évaluer :

1. L'impact du déséquilibre des classes ;
2. L'effet de l'optimisation des hyperparamètres ;
3. L'apport des techniques avancées de rééchantillonnage.

Modèles étudiés. Nous testons trois modèles :

- Régression logistique
- Random Forest
- XGBoost

Métrique d'évaluation. La métrique principale retenue pour la comparaison est le *F1-score*, particulièrement adaptée aux problèmes de classification déséquilibrée.

4.3 Tentative 1 : Modélisation sans traitement du déséquilibre

4.3.1 Régression Logistique

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import classification_report
3
4 model = LogisticRegression()
5
6 model.fit(X_train_encoded, y_train)
7
8 y_pred = model.predict(X_test_encoded)
9
10 report = classification_report(y_test, y_pred)
11
12 print(report)

```

	precision	recall	f1-score	support
0	0.97	0.99	0.98	11423
1	0.85	0.72	0.78	1074
accuracy			0.96	12497
macro avg	0.91	0.85	0.88	12497
weighted avg	0.96	0.96	0.96	12497

```

1 feature_importance = model.coef_[0]
2
3 # Cr er un DataFrame pour faciliter la manipulation
4 coef_df = pd.DataFrame(feature_importance,
5                          index=X_train_encoded.columns,
6                          columns=['Coefficients'])
7
8 # Trier les coefficients pour une meilleure visualisation
9 coef_df = coef_df.sort_values(by='Coefficients', ascending=
10                               True)
11
12 # Trac du graphique
13 plt.figure(figsize=(8, 4))
14 plt.barh(coef_df.index, coef_df['Coefficients'], color='
15           steelblue')
16 plt.xlabel('Valeur du coefficient')
17 plt.title('Importance des variables - R gression logistique'
18           )
19 plt.show()

```

avg_dpd_per_delinquency	residence_type_Owned	residence_type_Rented	loan_purpose_Education	loan_purpose_Home	loan_purpose_Personal	loan_type_Unsecured
0.590	True	False	False	True	False	False
0.620	True	False	True	False	False	False
0.560	False	False	False	True	False	False
0.000	False	False	True	False	False	False
0.000	True	False	False	False	True	True

4.3.2 Random Forest

```

1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier()
4
5 model.fit(X_train_encoded, y_train)
6
7 y_pred = model.predict(X_test_encoded)
8

```

```

9 report = classification_report(y_test, y_pred)
10
11 print(report)

```

	precision	recall	f1-score	support
0	0.97	0.99	0.98	11423
1	0.85	0.71	0.78	1074
accuracy			0.96	12497
macro avg	0.91	0.85	0.88	12497
weighted avg	0.96	0.96	0.96	12497

4.3.3 XGBoost

```

1 from xgboost import XGBClassifier
2
3 model = XGBClassifier()
4
5 model.fit(X_train_encoded, y_train)
6
7 y_pred = model.predict(X_test_encoded)
8
9 report = classification_report(y_test, y_pred)
10
11 print(report)

```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	11423
1	0.82	0.76	0.79	1074
accuracy			0.97	12497
macro avg	0.90	0.87	0.89	12497
weighted avg	0.96	0.97	0.96	12497

Comme il n'existe pas de différence significative de performance entre *XGBoost* et la *Régression Logistique*, nous retenons la *Régression Logistique* comme modèle candidat pour l'étape d'optimisation des hyperparamètres via *RandomizedSearchCV*.

Ce choix se justifie également par la meilleure interprétabilité de la régression logistique, ce qui est particulièrement important dans le contexte des modèles de scoring de crédit.

4.3.4 Optimisation des hyperparamètres – RandomizedSearchCV

Nous effectuons une recherche aléatoire sur les hyperparamètres suivants :

- Paramètre de régularisation C
- Solver
- Validation croisée à 3 plis
- Optimisation du F1-score

```
1 from sklearn.model_selection import RandomizedSearchCV
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.metrics import classification_report
4 import numpy as np
5
6
7 param_dist = {
8     'C': np.logspace(-4, 4, 20), # Valeurs de C espacées
9     'solver': ['lbfgs', 'saga', 'liblinear', 'newton-cg']
10 }
11
12 log_reg = LogisticRegression(max_iter=10000)
13
14
15 random_search = RandomizedSearchCV(
16     estimator=log_reg,
17     param_distributions=param_dist,
18     n_iter=50,
19     scoring='f1',
20     cv=3,
21     verbose=2,
22     random_state=42,
23     n_jobs=-1
24 )
25
26
27 random_search.fit(X_train_encoded, y_train)
28
29
30 print(f"Meilleurs param tres : {random_search.best_params_}"
31 )
32 print(f"Meilleur score : {random_search.best_score_}")
33
34 best_model = random_search.best_estimator_
35
36 y_pred = best_model.predict(X_test_encoded)
37
38
39 print("Rapport de classification :")
```

```
40 print(classification_report(y_test, y_pred))
```

Fitting 3 folds for each of 50 candidates, totalling 150 fits

Meilleurs param tres : {'solver': 'liblinear', 'C':
1438.44988828766}

Meilleur score : 0.7578820896729832

Rapport de classification :

	precision	recall	f1-score	support
0	0.98	0.99	0.98	11423
1	0.83	0.74	0.78	1074
accuracy			0.96	12497
macro avg	0.90	0.86	0.88	12497
weighted avg	0.96	0.96	0.96	12497

```
1 from scipy.stats import uniform, randint
2 from sklearn.model_selection import RandomizedSearchCV
3
4 # Define parameter distribution for RandomizedSearchCV
5 param_dist = {
6     'n_estimators': [100, 150, 200, 250, 300],
7     'max_depth': [3, 4, 5, 6, 7, 8, 9, 10],
8     'learning_rate': [0.01, 0.03, 0.05, 0.1, 0.15, 0.2, 0.25,
9     0.3],
10    'subsample': [0.6, 0.7, 0.8, 0.9, 1.0],
11    'colsample_bytree': [0.6, 0.7, 0.8, 0.9, 1.0],
12    'scale_pos_weight': [1, 2, 3, 5, 7, 10],
13    'reg_alpha': [0.01, 0.1, 0.5, 1.0, 5.0, 10.0],
14    'reg_lambda': [0.01, 0.1, 0.5, 1.0, 5.0, 10.0]
15 }
16 xgb = XGBClassifier()
17
18 random_search = RandomizedSearchCV(
19     estimator=xgb,
20     param_distributions=param_dist,
21     n_iter=100,
22     scoring='f1',
23     cv=3,
24     verbose=1,
25     n_jobs=-1,
26     random_state=42
27 )
28
29 random_search.fit(X_train_encoded, y_train)
```

```

30
31 # Print the best parameters and best score
32 print(f"Best Parameters: {random_search.best_params_}")
33 print(f"Best Score: {random_search.best_score_}")
34
35 best_model = random_search.best_estimator_
36
37 y_pred = best_model.predict(X_test_encoded)
38
39 print("Classification Report:")
40 print(classification_report(y_test, y_pred))

```

Fitting 3 folds for each of 100 candidates, totalling 300 fits

```

Best Parameters: {'subsample': 0.8, 'scale_pos_weight': 2, '
reg_lambda': 1.0,
'reg_alpha': 5.0, 'n_estimators': 100, 'max_depth': 3,
'learning_rate': 0.2, 'colsample_bytree': 0.9}

```

Best Score: 0.7883672970285227

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.98	0.98	11423
1	0.77	0.83	0.80	1074
accuracy			0.96	12497
macro avg	0.88	0.91	0.89	12497
weighted avg	0.97	0.96	0.96	12497

L'augmentation du nombre maximal d'itérations (`max_iter = 10000`) a permis d'assurer la convergence du modèle.

Cette étape a contribué à améliorer la robustesse du modèle initial.

4.4 Tentative 2 : Traitement du déséquilibre des classes

– Sous-échantillonnage (Random Under-Sampling)

Nous traitons le déséquilibre des classes par une méthode de *sous-échantillonnage aléatoire* (*Random Under-Sampling*).

```

1 from imblearn.under_sampling import RandomUnderSampler
2
3 rus = RandomUnderSampler(random_state=42)
4

```

```

5 X_train_res, y_train_res = rus.fit_resample(
6     X_train_encoded, y_train
7 )
8
9 y_train_res.value_counts()

```

```

default
0      3223
1      3223
Name: count, dtype: int64

```

Après l'application de la méthode de *Random Under-Sampling*, le jeu de données d'entraînement devient parfaitement équilibré.

Les deux classes de la variable cible (`default = 0` et `default = 1`) comptent désormais chacune 3 223 observations.

Ce rééquilibrage permet de réduire le biais du modèle en faveur de la classe majoritaire et d'améliorer sa capacité à détecter les clients en défaut de paiement.

```

1 model = LogisticRegression()
2
3 model.fit(X_train_res, y_train_res)
4
5 y_pred = model.predict(X_test_encoded)
6
7 report = classification_report(y_test, y_pred)
8
9 print(report)

```

	precision	recall	f1-score	support
0	1.00	0.91	0.95	11423
1	0.51	0.96	0.67	1074
accuracy			0.92	12497
macro avg	0.75	0.93	0.81	12497
weighted avg	0.95	0.92	0.93	12497

```

1 model = XGBClassifier(**random_search.best_params_)
2
3 model.fit(X_train_res, y_train_res)
4
5 y_pred = model.predict(X_test_encoded)
6
7 report = classification_report(y_test, y_pred)
8
9 print(report)

```


	precision	recall	f1-score	support
0	1.00	0.91	0.95	11423
1	0.51	0.99	0.67	1074
accuracy			0.92	12497
macro avg	0.75	0.95	0.81	12497
weighted avg	0.96	0.92	0.93	12497

Les résultats montrent une amélioration du *recall* pour la classe minoritaire (`default = 1`), ce qui signifie que le modèle détecte davantage de clients en situation de défaut.

Cependant, cette amélioration s'accompagne d'une légère dégradation de la précision et de la stabilité globale du modèle, en raison d'un plus grand nombre de faux positifs.

4.5 Tentative 3 : SMOTE Tomek + Optimisation Optuna

4.5.1 SMOTE Tomek

La méthode SMOTE Tomek combine :

- Sur-échantillonnage synthétique (*SMOTE*)
- Suppression des observations ambiguës (*Tomek links*)

```

1 from imblearn.combine import SMOTETomek
2
3 smt = SMOTETomek(random_state=42)
4
5 X_train_smt, y_train_smt = smt.fit_resample(X_train_encoded,
6       y_train)
7 y_train_smt.value_counts()
```

```

default
0      34195
1      34195
Name: count, dtype: int64
```

Objectifs de l'approche. Cette approche permet :

- d'équilibrer les classes ;
- de nettoyer les frontières décisionnelles ;
- d'améliorer la capacité de généralisation du modèle.

```

1 model = LogisticRegression()
2
3 model.fit(X_train_smt, y_train_smt)
4
5 y_pred = model.predict(X_test_encoded)
6
7 report = classification_report(y_test, y_pred)
8
9 print(report)

```

	precision	recall	f1-score	support
0	0.99	0.93	0.96	11423
1	0.55	0.94	0.70	1074
accuracy			0.93	12497
macro avg	0.77	0.94	0.83	12497
weighted avg	0.96	0.93	0.94	12497

4.5.2 4.5.2 Optimisation via Optuna

Nous avons utilisé *Optuna* pour optimiser automatiquement les hyperparamètres suivants :

- C
- tol
- $solver$
- $class_weight$

```

1 import optuna
2 from sklearn.metrics import make_scorer, f1_score
3 from sklearn.model_selection import cross_val_score
4
5
6
7
8
9
10

```

```

1 def objective(trial):
2     param = {
3         'C': trial.suggest_float('C', 1e-4, 1e4, log=True),
4         # Logarithmically spaced values
5         'solver': trial.suggest_categorical('solver', ['lbfgs',
6             'liblinear', 'saga', 'newton-cg']), # Solvers
7         'tol': trial.suggest_float('tol', 1e-6, 1e-1, log=True), # Logarithmically spaced values for tolerance
8         'class_weight': trial.suggest_categorical('class_weight', [None, 'balanced']) # Class weights
9     }
10     model = LogisticRegression(**param, max_iter=10000)

```

```

11
12 # Calculate the cross-validated f1_score
13 f1_scorer = make_scorer(f1_score, average='macro')
14 scores = cross_val_score(model, X_train_smt, y_train_smt,
15                             cv=3, scoring=f1_scorer, n_jobs=-1)
16
17 return np.mean(scores)
18
19 study_logistic = optuna.create_study(direction='maximize')
20 study_logistic.optimize(objective, n_trials=50)

```

```

[I 2026-03-04 12:45:25,287] A new study created in memory with name: no-name-63951202-b74b-4fb1-82f0-890808fddd13
[I 2026-03-04 12:45:27,013] Trial 0 finished with value: 0.9424221036946218 and parameters: {'C': 0.38101500444741393, 'solver': 'liblinear', 'tol': 0.0
414727031399081, 'class_weight': 'balanced'}. Best is trial 0 with value: 0.9424221036946218.
[I 2026-03-04 12:45:28,279] Trial 1 finished with value: 0.9454987880854588 and parameters: {'C': 1.3074164141113263, 'solver': 'liblinear', 'tol': 0.00
012419620144394356, 'class_weight': None}. Best is trial 1 with value: 0.9454987880854588.
[I 2026-03-04 12:45:30,763] Trial 2 finished with value: 0.9404145385567132 and parameters: {'C': 0.1337676849824779, 'solver': 'liblinear', 'tol': 0.00
8493218375401623, 'class_weight': 'balanced'}. Best is trial 1 with value: 0.9454987880854588.
[I 2026-03-04 12:45:31,047] Trial 3 finished with value: 0.9247657203848617 and parameters: {'C': 0.014141953584817929, 'solver': 'liblinear', 'tol': 4.
500891680412981e-05, 'class_weight': None}. Best is trial 1 with value: 0.9454987880854588.
[I 2026-03-04 12:45:31,200] Trial 4 finished with value: 0.9456192785931149 and parameters: {'C': 934.6818252266436, 'solver': 'newton-cg', 'tol': 0.000
2596232313445393, 'class_weight': 'balanced'}. Best is trial 4 with value: 0.9456192785931149.
[I 2026-03-04 12:45:31,462] Trial 5 finished with value: 0.8593447968977904 and parameters: {'C': 0.08151070864481526, 'solver': 'newton-cg', 'tol': 0.0
570454142215781, 'class_weight': 'balanced'}. Best is trial 4 with value: 0.9456192785931149.
[I 2026-03-04 12:45:31,799] Trial 6 finished with value: 0.9456061961958726 and parameters: {'C': 2706.475619760979, 'solver': 'lbfgs', 'tol': 0.0001149
383656389405, 'class_weight': None}. Best is trial 4 with value: 0.9456192785931149.
[I 2026-03-04 12:45:32,099] Trial 7 finished with value: 0.9456354400446974 and parameters: {'C': 84.06164038194899, 'solver': 'liblinear', 'tol': 4.196
97752929855e-06, 'class_weight': None}. Best is trial 7 with value: 0.9456354400446974.
[I 2026-03-04 12:45:32,310] Trial 8 finished with value: 0.9447011244418974 and parameters: {'C': 194.45453373808981, 'solver': 'liblinear', 'tol': 0.050
12565061508734, 'class_weight': 'balanced'}. Best is trial 7 with value: 0.9456354400446974.
[I 2026-03-04 12:45:32,518] Trial 9 finished with value: 0.9224226759747453 and parameters: {'C': 0.004113234453811339, 'solver': 'lbfgs', 'tol': 2.0190
807802248324e-06, 'class_weight': 'balanced'}. Best is trial 7 with value: 0.9456354400446974.
[I 2026-03-04 12:45:33,134] Trial 10 finished with value: 0.9456205698646887 and parameters: {'C': 17.707493693888193, 'solver': 'saga', 'tol': 1.654221
930177256e-06, 'class_weight': None}. Best is trial 7 with value: 0.9456354400446974.
[I 2026-03-04 12:45:33,725] Trial 11 finished with value: 0.9455914777982878 and parameters: {'C': 26.5287025283357, 'solver': 'saga', 'tol': 1.81063821
56536705e-06, 'class_weight': None}. Best is trial 7 with value: 0.9456354400446974.
[I 2026-03-04 12:45:34,391] Trial 12 finished with value: 0.9457227803228164 and parameters: {'C': 9.983105441021868, 'solver': 'saga', 'tol': 1.3348704
015110945e-05, 'class_weight': None}. Best is trial 12 with value: 0.9457227803228164.
[I 2026-03-04 12:45:34,983] Trial 13 finished with value: 0.8480346046161355 and parameters: {'C': 0.00021695290411727635, 'solver': 'saga', 'tol': 1.39
22564439313777e-05, 'class_weight': None}. Best is trial 12 with value: 0.9457227803228164.
[I 2026-03-04 12:45:35,550] Trial 14 finished with value: 0.9457665622389632 and parameters: {'C': 7.180345087667945, 'solver': 'saga', 'tol': 1.1150760
076249117e-05, 'class_weight': None}. Best is trial 14 with value: 0.9457665622389632.
[I 2026-03-04 12:45:36,233] Trial 15 finished with value: 0.9458242511864495 and parameters: {'C': 3.202059738631846, 'solver': 'saga', 'tol': 0.0007991
70243776589, 'class_weight': None}. Best is trial 15 with value: 0.9458242511864495.
[I 2026-03-04 12:45:36,600] Trial 16 finished with value: 0.9456024769276041 and parameters: {'C': 1.1010160792404184, 'solver': 'saga', 'tol': 0.001526
4443871921283, 'class_weight': None}. Best is trial 15 with value: 0.9458242511864495.
[I 2026-03-04 12:45:36,957] Trial 17 finished with value: 0.9458536928858772 and parameters: {'C': 3.2292642108653586, 'solver': 'saga', 'tol': 0.001977
892196159061, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:37,347] Trial 18 finished with value: 0.9457361347126559 and parameters: {'C': 2.9408165762625473, 'solver': 'saga', 'tol': 0.002265
891444832957, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:37,683] Trial 19 finished with value: 0.9456504032285403 and parameters: {'C': 229.0125993847762, 'solver': 'saga', 'tol': 0.0010698
85695628004, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:37,809] Trial 20 finished with value: 0.9205903775743255 and parameters: {'C': 0.025782690657447177, 'solver': 'lbfgs', 'tol': 0.008
213571513930627, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:38,175] Trial 21 finished with value: 0.9457089572151851 and parameters: {'C': 5.757653112926432, 'solver': 'saga', 'tol': 0.0005915
246127089983, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:38,492] Trial 22 finished with value: 0.9444131291424492 and parameters: {'C': 0.3400935489623467, 'solver': 'saga', 'tol': 0.000659
0278918016168, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:38,774] Trial 23 finished with value: 0.9456071353594081 and parameters: {'C': 9237.747855315109, 'solver': 'saga', 'tol': 0.00456349
9050812715, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:39,024] Trial 24 finished with value: 0.9455753753615262 and parameters: {'C': 41.72330036213603, 'solver': 'newton-cg', 'tol': 0.00
0513396839372753, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:39,424] Trial 25 finished with value: 0.9457076561279832 and parameters: {'C': 5.320538175409834, 'solver': 'saga', 'tol': 0.0002769
952563000797, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:39,627] Trial 26 finished with value: 0.9450857566261647 and parameters: {'C': 1.379800695277311, 'solver': 'saga', 'tol': 0.0170564
38661509275, 'class_weight': None}. Best is trial 17 with value: 0.9458536928858772.

```

```

42746416900247, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:42,217] Trial 33 finished with value: 0.9457792389698966 and parameters: {'C': 2.0609209515351754, 'solver': 'saga', 'tol': 0.000378
831972178431, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:42,495] Trial 34 finished with value: 0.9452359553584028 and parameters: {'C': 0.6617633849463612, 'solver': 'saga', 'tol': 0.004110
18541961348, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:42,914] Trial 35 finished with value: 0.9430619831198369 and parameters: {'C': 0.18740472451512805, 'solver': 'saga', 'tol': 0.00021
78969964271306, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:43,340] Trial 36 finished with value: 0.945621171141256 and parameters: {'C': 49.15430093671249, 'solver': 'saga', 'tol': 0.0012855
92856694743, 'class_weight': 'balanced'}). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:43,715] Trial 37 finished with value: 0.945664276083012 and parameters: {'C': 17.767573461490603, 'solver': 'liblinear', 'tol': 5.91
8623918631360e-05, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:44,000] Trial 38 finished with value: 0.9456466772116645 and parameters: {'C': 2.5511532706008797, 'solver': 'newton-cg', 'tol': 0.0
0020162736941524062, 'class_weight': 'balanced'}). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:44,289] Trial 39 finished with value: 0.9277246882735392 and parameters: {'C': 0.01928300459028399, 'solver': 'liblinear', 'tol': 0.
0017453573193591486, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:44,559] Trial 40 finished with value: 0.9453084493215981 and parameters: {'C': 0.6647437074770943, 'solver': 'lbfgs', 'tol': 8.06917
781278845e-05, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:45,088] Trial 41 finished with value: 0.9458529403834545 and parameters: {'C': 2.511931887453904, 'solver': 'saga', 'tol': 0.0004765
7252574909034, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:45,828] Trial 42 finished with value: 0.9456063494936031 and parameters: {'C': 89.78505455365385, 'solver': 'saga', 'tol': 0.0004756
667402041867, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:46,346] Trial 43 finished with value: 0.9457953892285009 and parameters: {'C': 4.1057245227828405, 'solver': 'saga', 'tol': 0.000165
0273119042097, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:46,835] Trial 44 finished with value: 0.9422843902046999 and parameters: {'C': 0.1501915167305862, 'solver': 'saga', 'tol': 2.8799341
44850645e-05, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:47,276] Trial 45 finished with value: 0.9383444669982378 and parameters: {'C': 0.048456487273991204, 'solver': 'saga', 'tol': 0.0001
4605959517048536, 'class_weight': 'balanced'}). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:47,679] Trial 46 finished with value: 0.945693479345837 and parameters: {'C': 15.669824791465093, 'solver': 'liblinear', 'tol': 2.51
0706060836903e-05, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:48,174] Trial 47 finished with value: 0.9458091046704915 and parameters: {'C': 2.5657026577458844, 'solver': 'saga', 'tol': 0.000329
505650269321, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:48,388] Trial 48 finished with value: 0.942139153372756 and parameters: {'C': 0.6644493480103415, 'solver': 'newton-cg', 'tol': 0.00
6475669140372106, 'class_weight': None). Best is trial 17 with value: 0.9458536928858772.
[I 2026-03-04 12:45:48,848] Trial 49 finished with value: 0.9458685882946879 and parameters: {'C': 3.4379368181400958, 'solver': 'saga', 'tol': 0.002199
002142147441, 'class_weight': 'balanced'}). Best is trial 49 with value: 0.9458685882946879.

```

```

1 print('Best trial:')
2 trial = study_logistic.best_trial
3
4 print('  F1-score: {}'.format(trial.value))
5 print('  Params: ')
6
7 for key, value in trial.params.items():
8     print('    {}: {}'.format(key, value))
9
10 best_model_logistic = LogisticRegression(**study_logistic.
    best_params)
11
12 best_model_logistic.fit(X_train_smt, y_train_smt)
13
14 # Evaluate on the test set
15 y_pred = best_model_logistic.predict(X_test_encoded)
16
17 report = classification_report(y_test, y_pred)
18
19 print(report)

```

```

Best trial:
  F1-score: 0.9458685882946879
  Params:
    C: 3.4379368181400958
    solver: saga
    tol: 0.002199002142147441
    class_weight: balanced

```

precision	recall	f1-score	support
-----------	--------	----------	---------

0	0.99	0.93	0.96	11423
1	0.56	0.94	0.70	1074
accuracy			0.93	12497
macro avg	0.77	0.94	0.83	12497
weighted avg	0.96	0.93	0.94	12497

Principe d'optimisation. La fonction objectif maximise le *F1-score macro* à l'aide d'une validation croisée.

Cette approche d'optimisation bayésienne s'est révélée plus efficace que la recherche aléatoire classique pour l'identification des meilleurs hyperparamètres.

4.6 Tentative 4 : XGBoost avec optimisation par Optuna

Motivation de l'approche. Bien que la régression logistique optimisée ait présenté d'excellentes performances ($AUC = 0.98$), il était méthodologiquement pertinent d'évaluer un modèle d'ensemble non linéaire optimisé dans les mêmes conditions expérimentales :

- Optimisation des hyperparamètres via *Optuna*
- Validation croisée ($CV = 3$)
- Optimisation du *F1-score macro*

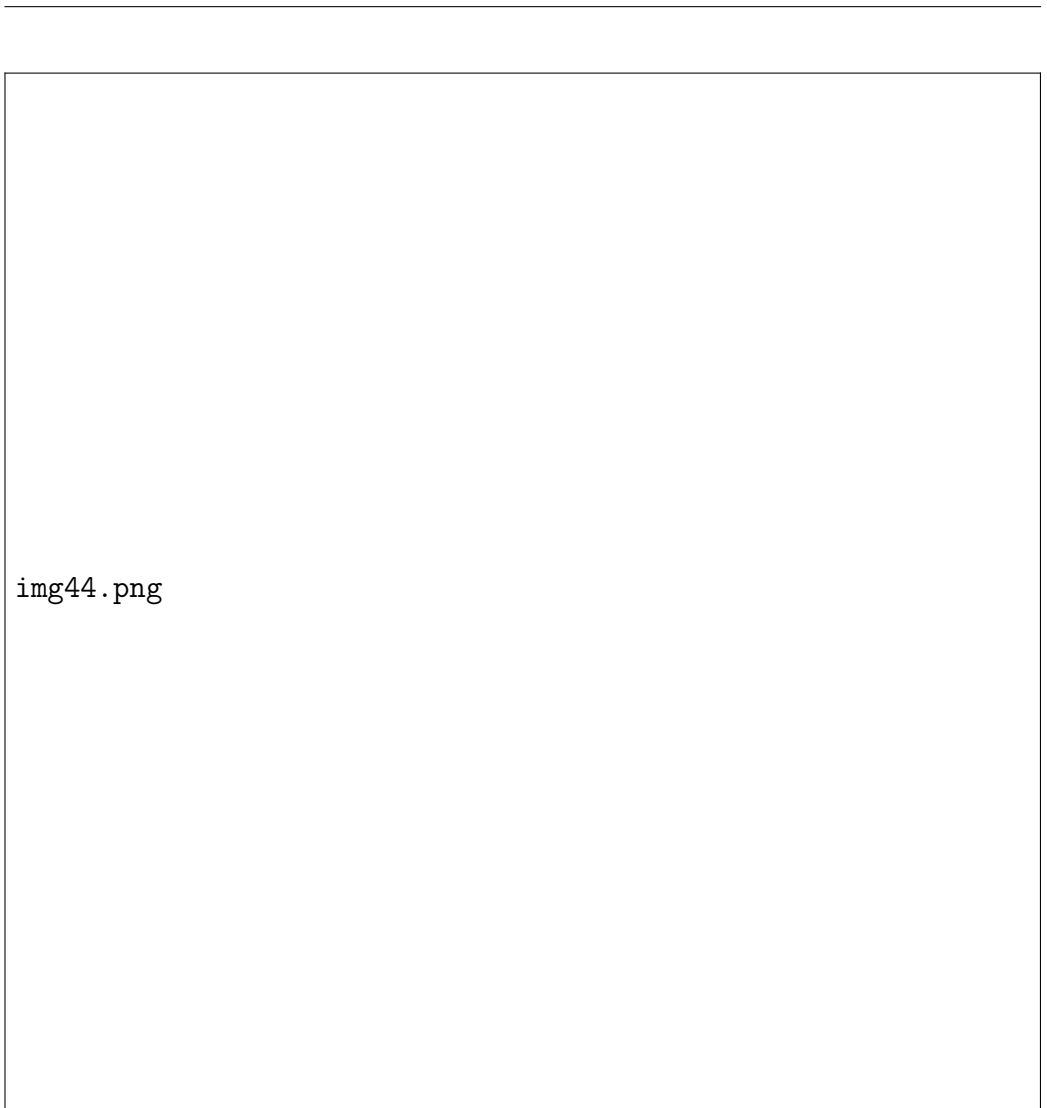
L'objectif était de vérifier si un modèle de *gradient boosting* pouvait surpasser la régression logistique tout en maintenant une bonne capacité de généralisation.

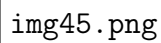
```

1 # Define the objective function for Optuna
2 def objective(trial):
3     param = {
4         'objective': 'binary:logistic',
5         'eval_metric': 'logloss',
6         'verbosity': 0,
7         'booster': 'gbtree',
8         'lambda': trial.suggest_float('lambda', 1e-3, 10.0,
9         log=True),
10        'alpha': trial.suggest_float('alpha', 1e-3, 10.0, log
11        =True),
12        'subsample': trial.suggest_float('subsample', 0.4,
13        1.0),
14        'colsample_bytree': trial.suggest_float('
15        colsample_bytree', 0.4, 1.0),
16        'max_depth': trial.suggest_int('max_depth', 3, 10),

```

```
13         'eta': trial.suggest_float('eta', 0.01, 0.3),
14         'gamma': trial.suggest_float('gamma', 0, 10),
15         'scale_pos_weight': trial.suggest_float('
scale_pos_weight', 1, 10),
16         'min_child_weight': trial.suggest_int('
min_child_weight', 1, 10),
17         'max_delta_step': trial.suggest_int('max_delta_step',
0, 10)
18     }
19
20     model = XGBClassifier(**param)
21
22     # Calculate the cross-validated f1_score
23     f1_scorer = make_scorer(f1_score, average='macro')
24     scores = cross_val_score(model, X_train_smt, y_train_smt,
25                             cv=3, scoring=f1_scorer, n_jobs
=-1)
26
27     return np.mean(scores)
28
29 study_xgb = optuna.create_study(direction='maximize')
30 study_xgb.optimize(objective, n_trials=50)
```





Résultat de l'optimisation. Après 50 essais, le meilleur *F1-score macro* obtenu en validation croisée est :

$$F1_{CV} = 0.9753$$

4.6.1 Évaluation sur le jeu de test

```
1 print('Best trial:')
2 trial = study_xgb.best_trial
3
4 print('  F1-score: {}'.format(trial.value))
5 print('  Params: ')
6
7 for key, value in trial.params.items():
```



```

8     print('    {}: {}'.format(key, value))
9
10    best_params = study_xgb.best_params
11
12    best_model_xgb = XGBClassifier(**best_params)
13
14    best_model_xgb.fit(X_train_smt, y_train_smt)
15
16    # Evaluate on the test set
17    y_pred = best_model_xgb.predict(X_test_encoded)
18
19    report = classification_report(y_test, y_pred)
20
21    print(report)

```

Listing 1 – Optimisation du modèle XGBoost avec Optuna et évaluation sur le jeu de test

```

Best trial:
F1-score: 0.9761463430816599
Params:
  lambda: 0.014205639805115234
  alpha: 0.3121468499521936
  subsample: 0.8380250670420222
  colsample_bytree: 0.9896457645089638
  max_depth: 9
  eta: 0.22854590177601766
  gamma: 0.4913347728621593
  scale_pos_weight: 1.7281669647387254
  min_child_weight: 7
  max_delta_step: 10

```

	precision	recall	f1-score	support
0	0.99	0.97	0.98	11423
1	0.72	0.86	0.79	1074
accuracy			0.96	12497
macro avg	0.86	0.92	0.88	12497
weighted avg	0.96	0.96	0.96	12497

Résultats globaux.

- Accuracy globale : 96 %
- F1 macro : 0.88

Analyse des résultats.

-
- Le modèle capture 86 % des défauts, ce qui constitue une performance particulièrement satisfaisante dans un contexte de gestion du risque de crédit.
 - La précision sur la classe minoritaire (0.72) indique la présence d'un certain nombre de faux positifs. Cette situation reste toutefois acceptable dans le contexte bancaire, où il est généralement préférable de refuser un client solvable plutôt que d'accepter un emprunteur présentant un risque élevé.
 - Le *F1-score macro* confirme une bonne stabilité des performances entre les deux classes.

4.7 Comparaison Logistique vs XGBoost

Critère	Régression Logistique	XGBoost
Interprétabilité	Très élevée	Faible
Capacité non-linéaire	Limitée	Élevée
F1-score (CV)	Très élevé	0.975
Recall classe défaut	Élevé	0.86
Complexité	Faible	Élevée
Temps d'entraînement	Rapide	Plus long

TABLE 8 – Comparaison des performances entre la régression logistique et XGBoost

Conclusion comparative. Malgré les excellentes performances obtenues par *XGBoost*, la *régression logistique* présente plusieurs avantages importants :

- Une performance quasi équivalente ;
- Une meilleure interprétabilité ;
- Une plus grande conformité aux standards réglementaires (*Bâle II/III*) ;
- Une robustesse plus facilement explicable.

Pour ces raisons, la régression logistique optimisée est retenue comme modèle final.

4.8 Évaluation approfondie du modèle final

4.8.1 Courbe ROC et AUC

```

1 y_pred = best_model_logistic.predict(X_test_encoded)
2
3 report = classification_report(y_test, y_pred)
4
5 print(report)

```

	precision	recall	f1-score	support
0	0.99	0.93	0.96	11423
1	0.56	0.94	0.70	1074
accuracy			0.93	12497
macro avg	0.78	0.94	0.83	12497
weighted avg	0.96	0.93	0.94	12497

```

1 from sklearn.metrics import roc_curve
2
3 probabilities = best_model_logistic.predict_proba(
4     X_test_encoded)[: ,1]
5
6 fpr, tpr, thresholds = roc_curve(y_test, probabilities)
7
8 fpr[:5], tpr[:5], thresholds[:5]

```

```

(array([0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
        8.75426771e-05,
        8.75426771e-05]),
array([0.          , 0.0009311 , 0.16294227, 0.16294227,
        0.17877095]),
array([          inf, 1.          , 0.99935556, 0.99931431,
        0.99920925]))

```

```

1 from sklearn.metrics import auc
2
3 area = auc(fpr, tpr)
4
5 area

```

```
0.9836982330562127
```

Cela signifie que le modèle distingue correctement un client en défaut d'un client sain dans environ 98 % des cas.

```

1
2 # Tracer la courbe ROC
3 plt.figure()
4

```

```

5 # Courbe ROC avec l'aire sous la courbe (AUC)
6 plt.plot(fpr, tpr, color='darkorange', lw=2,
7          label='Courbe ROC (aire = %0.2f)' % area)
8
9 # Ligne de r f r e n c e (mod le al atoire)
10 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
11
12 # Limites des axes
13 plt.xlim([0.0, 1.0])
14 plt.ylim([0.0, 1.05])
15
16 # Noms des axes
17 plt.xlabel('Taux de faux positifs')
18 plt.ylabel('Taux de vrais positifs')
19
20 # Titre du graphique
21 plt.title('Courbe ROC (Receiver Operating Characteristic)')
22
23 # Affichage de la l gende
24 plt.legend(loc="lower right")
25
26 # Affichage du graphique
27 plt.show()

```

4.8.2 Coefficient de Gini

Interprétation du coefficient de Gini. Le coefficient de Gini est compris entre -1 et 1 :

- une valeur proche de 1 indique un modèle (quasi) parfait ;
- une valeur proche de 0 indique un modèle sans pouvoir discriminant ;
- une valeur proche de -1 indique un modèle parfaitement incorrect (classement inversé).

```

1 gini_coefficient = 2 * area - 1
2
3 print("AUC:", area)
4 print("Gini Coefficient:", gini_coefficient)

```

```

AUC: 0.9836982330562127
Gini Coefficient: 0.9673964661124255

```

Une *AUC* de 0,98 indique que le modèle distingue très efficacement les événements (clients en défaut) des non-événements (clients non défaillants).

Le *coefficient de Gini* de 0,96 confirme cette performance en montrant que le modèle possède une très forte capacité de classement, proche d'une séparation parfaite entre les deux classes.

4.8.3 Rank Ordering (Déciles)

```
1 probabilities = best_model_logistic.predict_proba(  
    X_test_encoded)[: ,1]  
2  
3 df_eval = pd.DataFrame({  
4     'Default Truth': y_test,  
5     'Default Probability': probabilities  
6 })  
7  
8 df_eval.head(3)
```

	Default Truth	Default Probability
19205	0	0.549
15514	0	0.000
30367	0	0.007

```
1 df_eval['Decile'] = pd.qcut(  
2     df_eval['Default Probability'],  
3     10,  
4     labels=False,  
5     duplicates='drop'  
6 )  
7  
8 df_eval.head(4)
```

	Default Truth	Default Probability	Decile
19205	0	0.549	8
15514	0	0.000	2
30367	0	0.007	6
35347	0	0.007	6

```
1 df_eval[df_eval.Decile == 9]['Default Probability'].describe  
    ()
```

count	1250.000
mean	0.953
std	0.052
min	0.819
25%	0.922
50%	0.976
75%	0.997
max	1.000
Name:	Default Probability, dtype: float

Le décile 9 correspond aux clients présentant les probabilités de défaut les plus élevées selon le modèle.

La probabilité moyenne de défaut dans ce groupe est de 0,953, ce qui signifie que ces clients sont très fortement susceptibles de présenter un défaut de paiement. Les probabilités observées varient entre 0,819 et 1,000, confirmant que ce décile regroupe les profils les plus risqués.

Ces résultats montrent que le modèle parvient à classer efficacement les emprunteurs selon leur niveau de risque, en concentrant les probabilités de défaut les plus élevées dans les déciles supérieurs.

```

1 df_decile = (df_eval.groupby('Decile')
2     .agg(
3         Minimum_Probability=('Default Probability', 'min'),
4         Maximum_Probability=('Default Probability', 'max'),
5         Events=('Default Truth', 'sum'),
6         Non_events=('Default Truth', lambda s: s.size - s.sum
7     ))
8     .reset_index()
9 )
10
11 df_decile

```

Decile	Minimum_Probability	Maximum_Probability	Events	Non_events
0	0.000	0.000	0	1250
1	0.000	0.000	0	1250
2	0.000	0.000	0	1249
3	0.000	0.000	0	1250
4	0.000	0.001	0	1250
5	0.001	0.005	0	1249
6	0.005	0.031	5	1245
7	0.031	0.217	9	1240
8	0.217	0.818	161	1089
9	0.819	1.000	899	351

TABLE 9 – Répartition des événements et non-événements par décile de probabilité

```

1 df_decile['Event Rate'] = df_decile['Events']*100 / (
2     df_decile['Events'] + df_decile['Non_events'])
3 df_decile['Non-event Rate'] = df_decile['Non_events']*100 / (
4     df_decile['Events'] + df_decile['Non_events'])
5 df_decile

```

Decile	Min Prob.	Max Prob.	Events	Non-events	Event Rate	Non-event R
0	0.000	0.000	0	1250	0.000	100.000
1	0.000	0.000	0	1250	0.000	100.000
2	0.000	0.000	0	1249	0.000	100.000
3	0.000	0.000	0	1250	0.000	100.000
4	0.000	0.001	0	1250	0.000	100.000
5	0.001	0.005	0	1249	0.000	100.000
6	0.005	0.031	5	1245	0.400	99.600
7	0.031	0.217	9	1240	0.721	99.279
8	0.217	0.818	161	1089	12.880	87.120
9	0.819	1.000	899	351	71.920	28.080

TABLE 10 – Taux d'événements et de non-événements par décile de probabilité

```

1 df_decile = df_decile.sort_values(by='Decile', ascending=
    False).reset_index(drop=True)
2
3 df_decile

```

Decile	Min Prob.	Max Prob.	Events	Non-events	Event Rate	Non-event R
9	0.819	1.000	899	351	71.920	28.080
8	0.217	0.818	161	1089	12.880	87.120
7	0.031	0.217	9	1240	0.721	99.279
6	0.005	0.031	5	1245	0.400	99.600
5	0.001	0.005	0	1249	0.000	100.000
4	0.000	0.001	0	1250	0.000	100.000
3	0.000	0.000	0	1250	0.000	100.000
2	0.000	0.000	0	1249	0.000	100.000
1	0.000	0.000	0	1250	0.000	100.000
0	0.000	0.000	0	1250	0.000	100.000

TABLE 11 – Classement des déciles par probabilité décroissante

```

1 df_decile['Cum Events'] = df_decile['Events'].cumsum()
2 df_decile['Cum Non_events'] = df_decile['Non_events'].cumsum
  ()

```

```

3
4 df_decile

```

Decile	Min Prob.	Max Prob.	Events	Non-events	Event Rate	Non-event R
9	0.819	1.000	899	351	71.920	28.080
8	0.217	0.818	161	1089	12.880	87.120
7	0.031	0.217	9	1240	0.721	99.279
6	0.005	0.031	5	1245	0.400	99.600
5	0.001	0.005	0	1249	0.000	100.000
4	0.000	0.001	0	1250	0.000	100.000
3	0.000	0.000	0	1250	0.000	100.000
2	0.000	0.000	0	1249	0.000	100.000
1	0.000	0.000	0	1250	0.000	100.000
0	0.000	0.000	0	1250	0.000	100.000

TABLE 12 – Distribution cumulative des événements et non-événements par décile

```

1 df_decile['Cum Event Rate'] = df_decile['Cum Events'] * 100 /
  df_decile['Events'].sum()
2 df_decile['Cum Non-event Rate'] = df_decile['Cum Non_events']
  * 100 / df_decile['Non_events'].sum()
3
4 df_decile

```

Decile	Min_Prob	Max_Prob	Events	Non_events	Event_Rate	Non_event_Rate	Cum_Events	Cum_Non_eve
9	0.819	1.000	899	351	71.920	28.080	899	351
8	0.217	0.818	161	1089	12.880	87.120	1060	1440
7	0.031	0.217	9	1240	0.721	99.279	1069	2680
6	0.005	0.031	5	1245	0.400	99.600	1074	3925
5	0.001	0.005	0	1249	0.000	100.000	1074	5174
4	0.000	0.001	0	1250	0.000	100.000	1074	6424
3	0.000	0.000	0	1250	0.000	100.000	1074	7674
2	0.000	0.000	0	1249	0.000	100.000	1074	8923
1	0.000	0.000	0	1250	0.000	100.000	1074	10173
0	0.000	0.000	0	1250	0.000	100.000	1074	11423

TABLE 13 – Analyse par déciles des probabilités de défaut

Les deux premiers déciles concentrent près de 99 % des événements, ce qui indique que le modèle parvient à regrouper la grande majorité des défauts dans les segments de risque les plus élevés.

Le décile 9 présente :

- 72 % d'événements ;
- 28 % de non-événements.

Les déciles inférieurs ne contiennent aucun défaut, ce qui confirme une hiérarchisation extrêmement efficace des individus selon leur niveau de risque.

4.8.4 Statistique KS

Statistique KS. La statistique KS (Kolmogorov–Smirnov), qui mesure la différence maximale entre le taux cumulé d'événements et le taux cumulé de non-événements, atteint sa valeur maximale au **décile 8**, avec une valeur de **85,98 %**.

Cela indique que le modèle est particulièrement performant pour distinguer les événements des non-événements jusqu'à ce niveau de classement.

Au-delà de ce point, la valeur de la statistique KS diminue progressivement dans les déciles suivants, ce qui traduit une baisse graduelle de la capacité du modèle à séparer les observations selon leur niveau de risque.

```
1 df_decile['KS'] = abs(df_decile['Cum Event Rate'] - df_decile
2   ['Cum Non-event Rate'])
3 df_decile
```

[381]:

	Decile	Minimum_Probability	Maximum_Probability	Events	Non_events	Event Rate	Non-event Rate	Cum Events	Cum Non_events	Cum Event Rate	Cum Non-event Rate	KS
0	9	0.819	1.000	899	351	71.920	28.080	899	351	83.706	3.073	80.633
1	8	0.217	0.818	161	1089	12.880	87.120	1060	1440	98.696	12.606	86.090
2	7	0.031	0.217	9	1240	0.721	99.279	1069	2680	99.534	23.461	76.073
3	6	0.005	0.031	5	1245	0.400	99.600	1074	3925	100.000	34.361	65.639
4	5	0.001	0.005	0	1249	0.000	100.000	1074	5174	100.000	45.295	54.705
5	4	0.000	0.001	0	1250	0.000	100.000	1074	6424	100.000	56.237	43.763
6	3	0.000	0.000	0	1250	0.000	100.000	1074	7674	100.000	67.180	32.820
7	2	0.000	0.000	0	1249	0.000	100.000	1074	8923	100.000	78.114	21.886
8	1	0.000	0.000	0	1250	0.000	100.000	1074	10173	100.000	89.057	10.943
9	0	0.000	0.000	0	1250	0.000	100.000	1074	11423	100.000	100.000	0.000

Interprétation de la statistique KS. La valeur maximale de la statistique KS est de **85,98 %** et se situe au **décile 8**. Cela indique que la capacité du modèle à distinguer les événements des non-événements est la plus forte à ce niveau du classement.

En pratique, une règle empirique souvent utilisée consiste à considérer qu'un modèle est performant lorsque la valeur maximale de la statistique KS se situe dans les trois premiers déciles et que sa valeur dépasse **40 %**. Dans notre cas, avec une KS de **85,98 %**, le modèle présente donc un excellent pouvoir discriminant.

Interprétation de la table des déciles. Pour vérifier si l'ordre des rangs est respecté, on examine si les déciles supérieurs (ceux avec les probabilités prédites les plus élevées) présentent des taux d'événements plus élevés que les déciles inférieurs.

Un bon classement signifie qu'en passant du décile le plus élevé vers le décile le plus faible, le taux d'événements doit généralement diminuer.

Définitions :

— **Non-événements** : clients ne faisant pas défaut (bons clients).

— **Événements** : clients faisant défaut (clients risqués).

Les termes *événements* et *non-événements* peuvent être inversés selon le cas d'usage.

Exemple (marketing) :

— Les clients susceptibles d'accepter une offre ou un prêt peuvent être considérés comme des *événements*.

— Les clients qui n'accepteront pas l'offre seront considérés comme des *non-événements*.

Analyse des déciles Déciles supérieurs :

— **Décile 9** : taux d'événements élevé (72 %) et taux de non-événements de 28 %. Cela indique que le modèle est très confiant dans la prédiction des défauts dans ce segment.

— **Décile 8** : taux d'événements également significatif (12,72 %), avec un taux d'événements cumulés atteignant environ 98,6 %.

Déciles intermédiaires :

— **Déciles 7 et 6** : forte diminution des taux d'événements, ce qui confirme la capacité du modèle à discriminer les niveaux de risque.

Déciles inférieurs :

— **Déciles 5 à 0** : aucun événement observé. Toutes les observations correspondent à des non-événements.

Ces déciles présentent donc collectivement un taux de non-événements de 100 %, ce qui confirme l'efficacité du modèle dans la hiérarchisation du risque de défaut.

4.9 Importance des variables

```
1 final_model = best_model_logistic
2
3 feature_importance = final_model.coef_[0]
4
5 # Cr er un DataFrame pour faciliter la manipulation
```

```

6 coef_df = pd.DataFrame(feature_importance, index=
    X_train_encoded.columns, columns=['Coefficients'])
7
8 # Trier les coefficients pour une meilleure visualisation
9 coef_df = coef_df.sort_values(by='Coefficients', ascending=
    True)
10
11 # Trac du graphique
12 plt.figure(figsize=(8, 4))
13 plt.barh(coef_df.index, coef_df['Coefficients'], color='
    steelblue')
14 plt.xlabel('Valeur du coefficient')
15 plt.title("Importance des variables - R gression logistique"
    )
16 plt.show()

```

[381]:

	Decile	Minimum_Probability	Maximum_Probability	Events	Non_events	Event Rate	Non-event Rate	Cum Events	Cum Non_events	Cum Event Rate	Cum Non-event Rate	KS
0	9	0.819	1.000	899	351	71.920	28.080	899	351	83.706	3.073	80.633
1	8	0.217	0.818	161	1089	12.880	87.120	1060	1440	98.696	12.606	86.090
2	7	0.031	0.217	9	1240	0.721	99.279	1069	2680	99.534	23.461	76.073
3	6	0.005	0.031	5	1245	0.400	99.600	1074	3925	100.000	34.361	65.639
4	5	0.001	0.005	0	1249	0.000	100.000	1074	5174	100.000	45.295	54.705
5	4	0.000	0.001	0	1250	0.000	100.000	1074	6424	100.000	56.237	43.763
6	3	0.000	0.000	0	1250	0.000	100.000	1074	7674	100.000	67.180	32.820
7	2	0.000	0.000	0	1249	0.000	100.000	1074	8923	100.000	78.114	21.886
8	1	0.000	0.000	0	1250	0.000	100.000	1074	10173	100.000	89.057	10.943
9	0	0.000	0.000	0	1250	0.000	100.000	1074	11423	100.000	100.000	0.000

Interprétation. L'analyse des coefficients montre que les variables dominantes sont :

- loan_to_income
- delinquency_ratio
- credit_utilization_ratio

Ces variables, liées au comportement financier et à la solvabilité des clients, apparaissent comme les plus influentes dans la prédiction du risque de défaut.

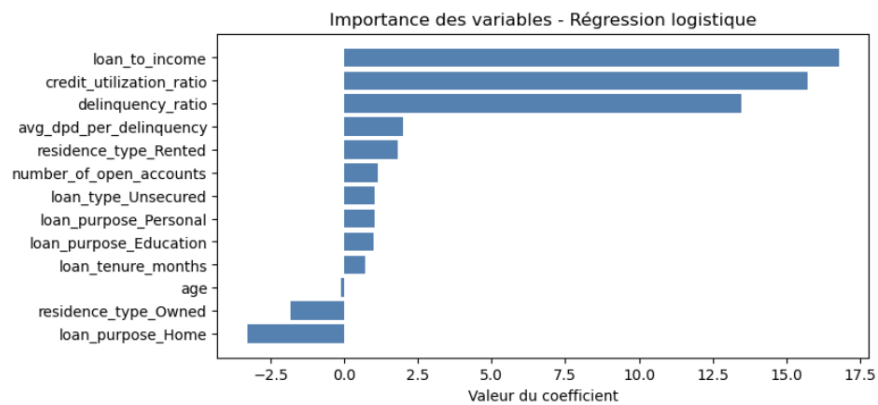
Autrement dit, elles contribuent fortement à déterminer si un client est susceptible de rembourser ou non son crédit.

4.10 Industrialisation

```

1 X_test_encoded.head(2)

```



[387]:

	age	loan_tenure_months	number_of_open_accounts	credit_utilization_ratio	loan_to_income	delinquency_ratio	avg_dpd_per_delinquency	residence_type_Owned
19205	0.346	0.755	0.333	0.990	0.550	0.000	0.000	True
15514	0.481	0.226	1.000	0.323	0.220	0.000	0.000	True

```
1 X_test_encoded.columns
```

```
Index(['age', 'loan_tenure_months', 'number_of_open_accounts',
      'credit_utilization_ratio', 'loan_to_income', 'delinquency_ratio',
      'avg_dpd_per_delinquency', 'residence_type_Owned',
      'residence_type_Rented', 'loan_purpose_Education', 'loan_purpose_Home',
      'loan_purpose_Personal', 'loan_type_Unsecured'],
      dtype='object')
```

```
1 from joblib import dump
2
3 model_data = {
4     'model': final_model,
5     'features': X_train_encoded.columns,
6     'scaler': scaler,
7     'cols_to_scale': cols_to_scale
8 }
9
10 dump(model_data, 'model_data.joblib')
```

```
['model_data.joblib']
```

```
1 final_model.coef_, final_model.intercept_
```

```
(array([[ -0.12679496,  0.71147603,  1.14967869, 15.72508479,
         16.80030325,
          13.48014734,  2.01202819, -1.82153578,  1.81465734,
          0.98970051,
          -3.29437867,  1.01443429,  1.01443429]]),
 array([ -20.37591994]))
```

Le modèle final a été sauvegardé avec :

- le modèle entraîné ;
- la liste des variables utilisées.

Cette étape garantit :

- la reproductibilité des résultats ;
- la possibilité de déploiement du modèle ;
- la traçabilité du processus de modélisation.

Conclusion

Ce projet avait pour objectif de développer un modèle prédictif capable d'estimer le risque de défaut de crédit en exploitant des techniques modernes de Machine Learning appliquées à des données financières. L'approche adoptée s'est articulée autour de plusieurs étapes essentielles : préparation des données, exploration statistique, entraînement de différents modèles de classification et évaluation approfondie de leurs performances.

Les résultats obtenus mettent en évidence des performances particulièrement élevées du modèle retenu. En effet, l'évaluation montre une capacité discriminante très importante, avec une AUC de 0,98, indiquant que le modèle distingue très efficacement les clients susceptibles de faire défaut de ceux présentant un faible risque. Cette performance est confirmée par un coefficient de Gini de 0,96, traduisant un pouvoir de classement quasi parfait des profils de risque. De plus, la statistique KS de 85,98 % met en évidence une séparation très nette entre les événements (défauts) et les non-événements, ce qui constitue un indicateur fort de la qualité du modèle dans un contexte de scoring de crédit.

L'analyse des coefficients révèle que les variables les plus déterminantes dans la prédiction du défaut sont le ratio prêt/revenu (`loan_to_income`), le ratio d'incidents de paiement (`delinquency_ratio`) et le taux d'utilisation du crédit (`credit_utilization_ratio`). Ces variables reflètent principalement le niveau d'endettement et le comportement financier des clients, deux facteurs clés dans l'évaluation du risque de crédit.

Parmi les modèles testés, la régression logistique optimisée et entraînée sur un jeu de données équilibré à l'aide de la méthode SMOTE Tomek s'est

révélée être la solution la plus adaptée. Ce choix repose sur un compromis particulièrement intéressant entre performance prédictive, robustesse statistique et interprétabilité du modèle, ce dernier point étant essentiel dans le contexte bancaire où la transparence des décisions algorithmiques est souvent requise pour des raisons réglementaires et opérationnelles.

Cependant, un niveau de performance aussi élevé nécessite une interprétation prudente. Plusieurs limites doivent être prises en compte, notamment le risque potentiel de surapprentissage, qui pourrait réduire la capacité de généralisation du modèle sur de nouvelles données. De plus, une validation sur des jeux de données externes serait nécessaire afin de confirmer la robustesse du modèle dans un contexte réel. Enfin, dans une perspective de déploiement opérationnel, il serait indispensable de mettre en place un système de suivi et de monitoring du modèle, afin de détecter d'éventuelles dérives de performance liées à l'évolution des comportements financiers ou des conditions économiques.

Au-delà de ses résultats techniques, ce projet illustre de manière concrète la contribution de la Data Science à la gestion du risque bancaire. Il montre que les méthodes de Machine Learning peuvent constituer des outils puissants pour améliorer la prise de décision financière, à condition d'être intégrées dans une démarche méthodologique rigoureuse combinant qualité des données, expertise métier et validation statistique approfondie.

Ainsi, la modélisation du risque de crédit apparaît comme un domaine emblématique de la convergence entre finance et science des données. La valeur d'un modèle ne réside pas uniquement dans sa précision statistique, mais également dans sa robustesse, sa transparence et sa capacité à être intégré dans des processus décisionnels réels.

Ce projet démontre qu'une approche structurée de l'analyse des données et de la modélisation peut contribuer de manière significative à l'amélioration des systèmes de scoring de crédit et de gestion du risque financier.

5 Références

- Notes de cours Master M1 SAAD , Université de Caen M.Christophe Chesneau <https://sites.google.com/view/chesneau/enseignements-s2?authuser=0>
- Notes de cours Master M2 SAAD , Université de Caen M.Christophe Chesneau
- Notes de cours Master M2 SAAD , Université de Caen M.Bertrand Maillot
- wikipedia